



Intensivão de **JAVASCRIPT**

T

Apostila Completa

AULA 1

Guia passo a passo: Projeto Aplicativo de Audiobook



Parte 1

Instalando as ferramentas



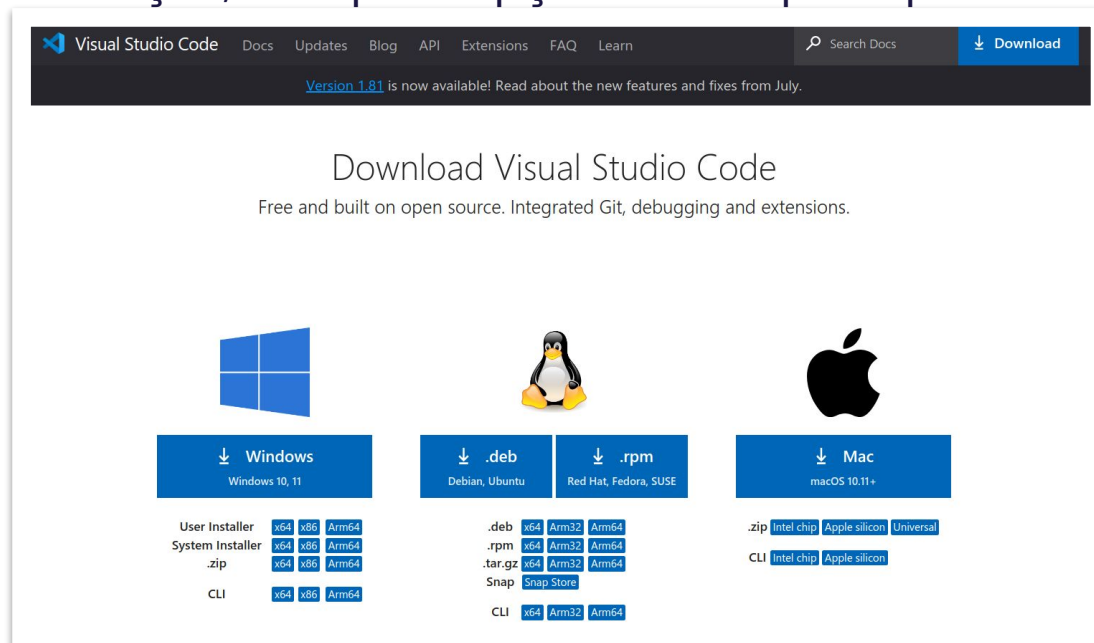


Instalando as ferramentas – VS CODE

Caso você ainda não tenha o Visual Studio Code instalado, basta seguir os procedimentos abaixo. A instalação do VS Code é totalmente gratuita, e você pode usá-lo em seu computador sem precisar pagar nada. O link para fazer o download do programa é mostrado abaixo:

<https://code.visualstudio.com/>

- 1 – Baixe o arquivo de instalação correspondente ao seu sistema operacional (Windows, MacOS ou Linux).
- 2 – Execute o arquivo de instalação e siga as instruções na tela. Em geral, é só continuar clicando em "Próximo".
- 3 – No Windows, durante a instalação, marque a opção "Add to path" para adicionar o VS Code às suas variáveis de ambiente.



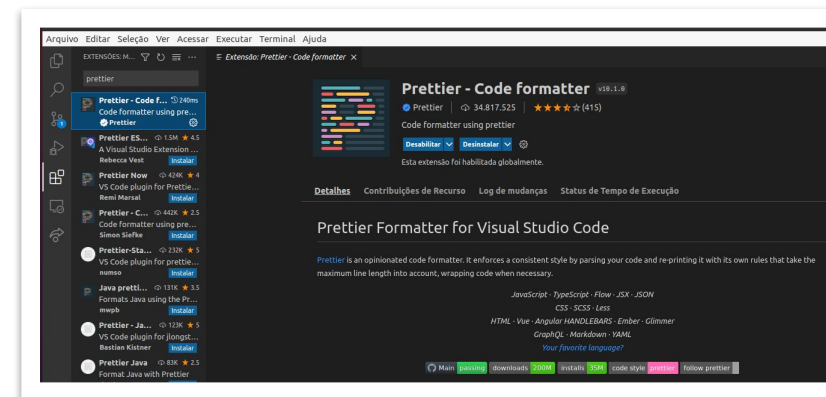


Instalando as ferramentas – VS CODE - EXTENSÕES

As extensões no Visual Studio Code são pequenos programas que adicionam recursos extras ao editor de código. Elas são como "plugins" que podem ser instalados para personalizar e estender as funcionalidades do VS Code.

A extensão Prettier adiciona a capacidade de formatar automaticamente o código, tornando-o mais legível e organizado.

Para instalar a extensão Prettier no Visual Studio Code:



- Clique no ícone de extensões no menu lateral esquerdo (ou use o atalho "Ctrl+Shift+X").
- Na barra de pesquisa, digite "Prettier".
- A extensão "Prettier - Code Formatter" deve aparecer nos resultados da pesquisa. Clique no botão "Instalar" ao lado dela.
- Após a instalação, você pode configurar o Prettier como o formatador padrão para o seu projeto. Para fazer isso, abra as configurações do Visual Studio Code (menu "File" > "Preferences" > "Settings" ou use o atalho "Ctrl+,").
- Na barra de pesquisa das configurações, digite "Default Formatter" e selecione a opção "Editor: Default Formatter".
- No campo de valor, digite "esbenp.prettier-vscode" e pressione Enter para salvar as alterações.
- Agora, o Prettier está instalado e configurado como o formatador padrão no Visual Studio Code. Você pode usar o comando "Format Document" (menu "Edit" > "Format Document" ou atalho "Shift+Alt+F") para formatar o código.



Instalando as ferramentas – VS CODE - EXTENSÕES

Já a extensão ES6 String HTML adiciona suporte para destacar a sintaxe do HTML dentro de strings de várias linhas no código JavaScript.

Para instalar a extensão ES6 String HTML no Visual Studio Code:

- Clique no ícone de extensões no menu lateral esquerdo (ou use o atalho "Ctrl+Shift+X").
- Na barra de pesquisa, digite "ES6 String HTML".
- A extensão "ES6 String HTML" deve aparecer nos resultados da pesquisa. Clique no botão "Instalar" ao lado dela.
- Após a instalação, a extensão irá adicionar suporte de realce de sintaxe para código HTML dentro de strings de várias linhas no ES6.



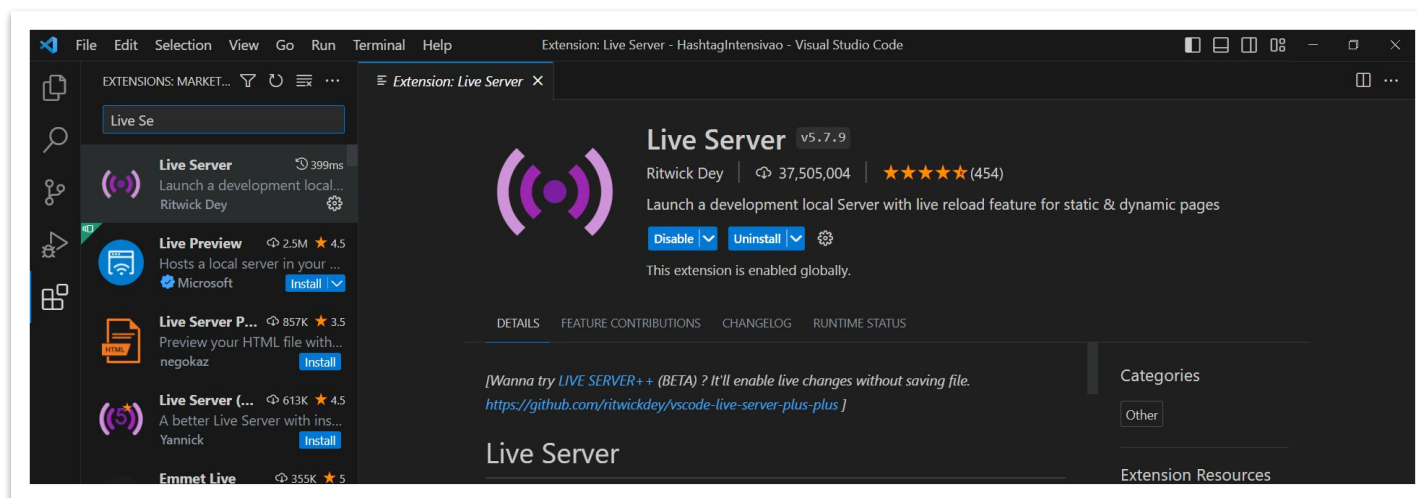


Instalando as ferramentas – VS CODE - EXTENSÕES

O Live Server é uma extensão muito útil para desenvolvimento web, pois facilita a visualização e a atualização instantânea das alterações feitas no código HTML.

Vou te explicar como baixar a extensão "Live Server" no VS Code:

- Abra o Visual Studio Code (VS Code) e vá para a seção de extensões.
- Pesquise por "Live Server" e clique em "Instalar" ao lado da extensão desenvolvida por Ritwick Dey.
- Aguarde a instalação e clique em "Gerenciar" para acessar as configurações da extensão.
- Personalize as configurações, se desejar.
- Abra um arquivo HTML no VS Code, clique com o botão direito do mouse e selecione "Abrir com Live Server".
- O Live Server iniciará um servidor local e abrirá o arquivo HTML no navegador padrão.





Instalando as ferramentas – Node.js

O Node.js é um ambiente de execução de código JavaScript do lado do servidor. Ele permite que você execute código JavaScript fora do navegador, o que significa que você pode criar aplicativos de servidor, scripts de linha de comando e muito mais usando JavaScript. Para instalar o Node.js, você pode seguir os seguintes passos:



- Acesse o site oficial do Node.js em <https://nodejs.org>.
- Na página inicial, você verá duas versões para download: LTS (Long Term Support) e Current. A versão LTS é recomendada para a maioria dos usuários, pois é mais estável e possui suporte a longo prazo. Selecione a versão LTS ou a versão mais recente, se preferir.
- Após selecionar a versão desejada, você será redirecionado para a página de download. Escolha o instalador adequado para o seu sistema operacional (Windows, macOS ou Linux) e clique no link para iniciar o download.
- Após o download ser concluído, execute o instalador e siga as instruções na tela para concluir a instalação.
- Após a instalação ser concluída, você pode verificar se o Node.js foi instalado corretamente abrindo o terminal ou prompt de comando e digitando o comando `node -v`. Se a versão do Node.js for exibida, significa que a instalação foi bem-sucedida.

Parte 2

Apresentação do Projeto Aplicativo de Audiobook!

Apresentação do Projeto Aplicativo de Audiobook!

Bem-vindos ao nosso projeto de **Aplicativo Audiobook**, onde exploraremos a implementação de uma plataforma interativa para desfrutar da obra-prima literária "Dom Casmurro" em formato de áudio. Alinhado à proposta de sites de audiobook, nosso projeto oferecerá aos usuários acesso a áudios previamente preparados. Inspirados pelo conceito de plataformas de audiobook online, nosso objetivo é proporcionar uma experiência envolvente, permitindo que os usuários explorem uma diversificada seleção de áudios, desfrutando das narrativas sem a necessidade de leitura convencional. Com o uso do JavaScript, destacamos como a programação web pode criar uma plataforma dinâmica e acessível, facilitando a imersão na riqueza da obra de Machado de Assis.

Dentro deste PDF, você encontrará um guia apresentando o passo a passo para criar o projeto completo. O conteúdo está incrível e acredito que você vai curtir cada detalhe!



Parte 3

Tríade - HTML, CSS E Javascript

Tríade - HTML, CSS e Javascript

Bem-vindos à nossa aula sobre a criação de um audiobook do zero! Antes de mergulharmos no fascinante mundo da narração sonora, é crucial compreendermos a interação entre HTML, CSS e JavaScript – as ferramentas fundamentais que utilizaremos para dar vida ao nosso projeto. Essas três linguagens trabalham em conjunto para criar experiências web dinâmicas e envolventes.

- O **HTML (Hypertext Markup Language)** é a espinha dorsal do nosso conteúdo. Ele estrutura a informação, definindo os elementos que compõem a nossa página, como títulos, parágrafos, imagens e links. Imagine o HTML como o esqueleto do corpo, proporcionando a base necessária para a construção do conteúdo.
- O **CSS (Cascading Style Sheets)** entra em cena para adicionar estilo e estética à nossa página HTML. Enquanto o HTML fornece a estrutura, o CSS permite a personalização visual, aplicando cores, fontes, margens e outros estilos para criar uma experiência atraente e coesa. Analogamente, o CSS é como a roupa que vestimos sobre o esqueleto, tornando a aparência mais agradável e organizada.

Tríade - HTML, CSS e Javascript

- **JavaScript** é a linguagem de programação que dá vida à nossa experiência de audiobook, tornando-a interativa e dinâmica. Essencial para manipular HTML e CSS, o JavaScript entra em cena para adicionar, remover ou modificar conteúdo, respondendo às ações do usuário de forma inteligente. Em nosso projeto, ele não apenas gerencia a interface, mas também é responsável pela reprodução de áudio. Ao responder a cliques em botões ou comandos específicos, o JavaScript se torna o maestro por trás da cortina, coordenando a experiência auditiva de forma envolvente.

Juntos, HTML, CSS e JavaScript formam a tríade essencial para a criação de páginas web interativas e atraentes. Ao longo desta aula, exploraremos como essas linguagens se complementam para dar vida ao nosso projeto de audiobook, proporcionando não apenas uma experiência auditiva memorável, mas também uma compreensão mais profunda de como essas ferramentas se entrelaçam para criar a magia da web moderna. Vamos começar a jornada da criação do nosso audiobook!

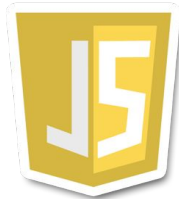


Parte 4

0 Javascript

O Javascript

O que é JavaScript?



JavaScript é uma linguagem de programação de alto nível, dinâmica e interpretada, que é usada principalmente para tornar as páginas da web interativas. Com JavaScript, os desenvolvedores podem controlar o comportamento de elementos da página da web, criar animações, processar e manipular dados, entre outras coisas. A linguagem é executada no navegador do cliente, o que significa que o código é executado no dispositivo do usuário, em vez do servidor web, tornando a interação do usuário com a página da web mais rápida e eficiente.

O JavaScript desempenha um papel crucial na dinâmica e interatividade do nosso projeto de audiobook. Em essência ao ser incorporada nas páginas web, permite aos desenvolvedores controlar e aprimorar a experiência do usuário de maneira personalizada e eficiente.

Originalmente concebido em 1995 por Brendan Eich, o JavaScript foi criado para trazer animações e alertas às páginas da web. Ao longo do tempo, sua popularidade cresceu, e com o suporte da Microsoft em 1996, tornou-se uma linguagem-chave na programação web.

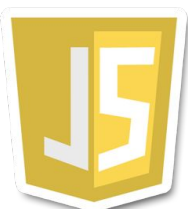
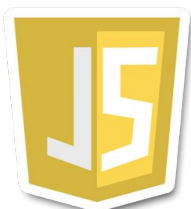
O Javascript

A evolução do JavaScript trouxe o ES6 (ECMAScript 2015), uma versão que introduziu recursos avançados, como declarações de variáveis mais flexíveis (let e const), classes, módulos e promessas. Esses aprimoramentos elevaram o JavaScript a uma linguagem mais poderosa e amigável.

Quando aplicamos o JavaScript ao nosso projeto de audiobook, ele se torna a força motriz por trás da interface interativa. Responsável por manipular HTML e CSS, o JavaScript permite a atualização dinâmica da interface do usuário. Ele opera no lado do cliente, garantindo uma resposta rápida e eficiente, sem depender constantemente do servidor.

No contexto específico do audiobook, o JavaScript assume o papel de maestro, controlando a reprodução de áudio, respondendo a interações do usuário, e adicionando camadas de dinamismo à experiência de audição. Com ele, podemos oferecer aos usuários a capacidade de controlar a reprodução, pular entre capítulos e interagir de maneira intuitiva com o conteúdo sonoro.

Em resumo, a implementação do JavaScript no nosso projeto de audiobook proporciona não apenas uma experiência interativa, mas também adiciona camadas de dinamismo e controle aos elementos sonoros, elevando a narrativa a uma experiência envolvente e personalizada para os usuários.



Parte 5

Primeiros Passos

Primeiros Passos

Para iniciar o nosso projeto, vamos fazer o download da pasta com o nome "Do zero" ou você pode criar uma pasta inicial no seu computador. Essa pasta será o local onde iremos armazenar todos os arquivos do nosso audiobook. Dentro dessa pasta, colocaremos os arquivos HTML, CSS e Javascript, além de alguns arquivos adicionais que serão utilizados, como os arquivos de áudio do livro "Dom Casmurro" e imagem .

Vou te explicar passo a passo como criar uma pasta nova no Windows:

- Primeiro, abra o "Explorador de Arquivos" clicando no ícone da pasta amarela na barra de tarefas ou pressionando a tecla "Windows" + "E" no teclado.
- Navegue até o local onde você deseja criar a nova pasta. Por exemplo, você pode escolher criar a pasta na área de trabalho ou dentro de outra pasta existente.
- Com o local selecionado, clique com o botão direito do mouse em um espaço vazio da janela do Explorador de Arquivos. Um menu de opções será exibido.

Primeiros Passos

- No menu de opções, passe o cursor sobre a opção "Novo" e, em seguida, clique em "Pasta". Uma nova pasta será criada com o nome "Nova pasta" destacado.
- Digite o nome desejado para a nova pasta. No nosso caso, digite "Intensivao" como o nome da pasta.
- Pressione a tecla "Enter" no teclado para confirmar o nome da pasta. A nova pasta será criada no local selecionado.



Nome	Status	Data de modificação	Tipo	Tamanho
 Intensivao		26/09/2025 06:52	Pasta de arquivos	

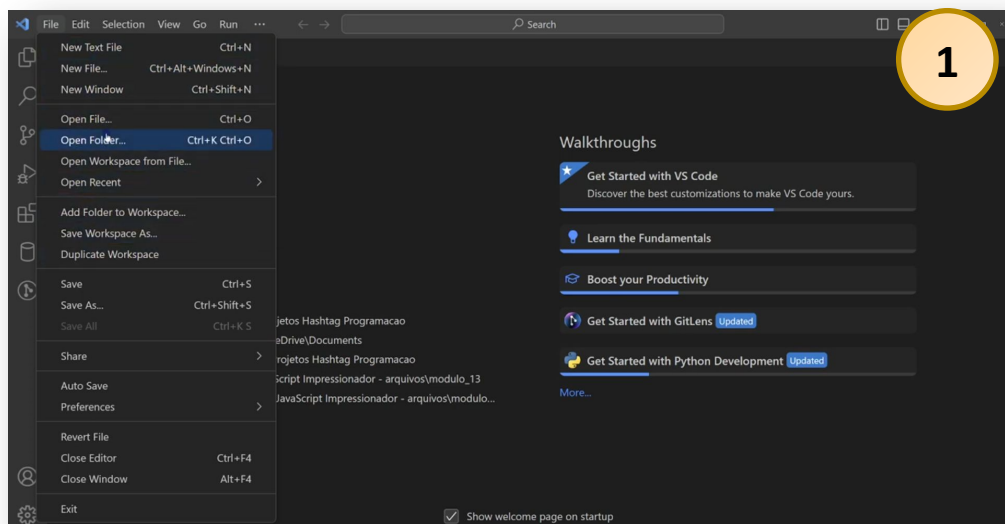
Primeiros Passos

Agora vamos abrir a nossa pasta no VS Code, que será o programa que utilizaremos para realizarmos a construção do nosso audiobook. Para fazer isso, siga os passos abaixo:

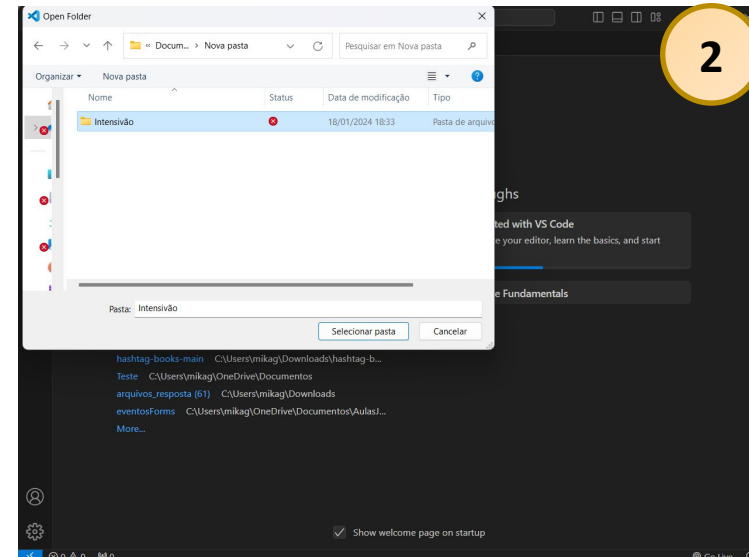
- Certifique-se de ter o Visual Studio Code instalado no seu computador. Se ainda não tiver, você pode baixá-lo gratuitamente no site oficial do VS Code.
- Abra o Visual Studio Code clicando no ícone do programa na área de trabalho ou no menu Iniciar.
- No VS Code, clique em 'Arquivo' no menu superior e, em seguida, selecione 'Abrir Pasta'. Uma janela de seleção de pasta será exibida **(Imagem 1)**.
- Navegue até o local onde você criou a pasta 'Intensivão'. Por exemplo, se você a criou na área de trabalho, vá até a área de trabalho **(Imagem 2)**.
- Selecione a pasta 'Intensivao' e clique no botão 'Selecionar Pasta' na parte inferior direita da janela.
- A pasta 'Intensivao' será aberta no VS Code. No painel lateral esquerdo, você encontrará o local específico para criar os arquivos do seu projeto **(Imagem 3)**.

Primeiros Passos

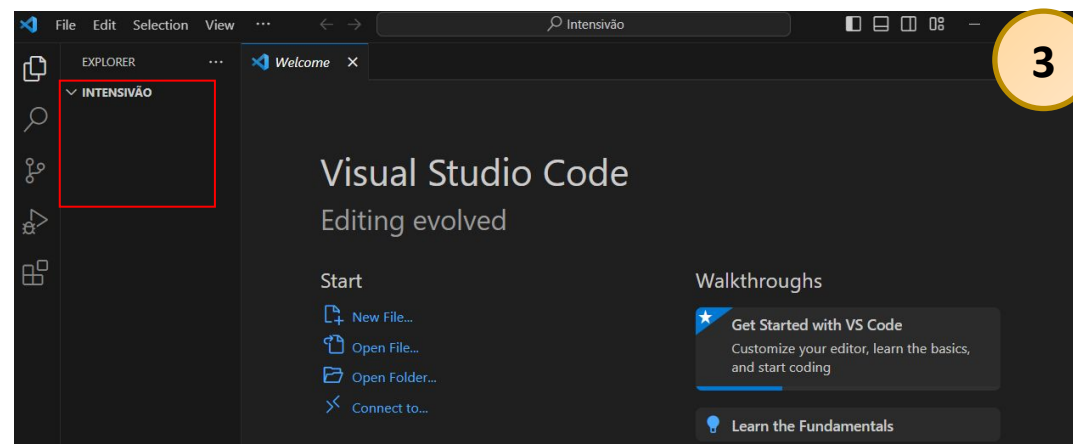
- Abrir Folder/Pasta



- Procurando pasta criada para projeto



- Local dos Arquivos e Pastas do projeto



Primeiros Passos

Passo a passo – baixar, descompactar e abrir a pasta DoZero.

1. Fazer o download

 DoZero	25/09/2025 21:10	Pasta compactada	92.853 KB
--	------------------	------------------	-----------

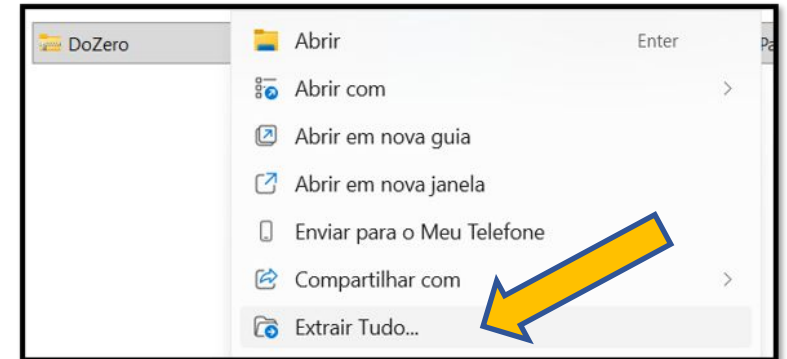
- Clique no link fornecido para o material e escolha Salvar (geralmente vai para a pasta Downloads do seu computador).

- Confirme que o arquivo baixado se chama algo como **DoZero.zip**

2. Localizar o arquivo baixado

- Abra a pasta Downloads (ou a pasta onde você salvou o arquivo).

- Verifique o nome do arquivo **.zip**.



3. Extrair (descompactar) o arquivo ZIP

- Clique com o botão direito sobre **DoZero.zip** → Extrair tudo... → escolha onde salvar (por exemplo, Área de Trabalho) → Extrair.

Primeiros Passos

4. Verificar o conteúdo extraído

- Abra a pasta extraída e confirme que ela contém os arquivos do projeto (ex.: **pasta audios**, **pasta imagens**).
- Atenção a **pasta aninhada**: às vezes o ZIP cria **DoZero/DoZero/**. Se isso acontecer, mova a pasta com os arquivos para o local desejado (ex.: Área de Trabalho) ou abra a pasta interna que contém os arquivos do projeto.



5. Mover a pasta para o local desejado (opcional)

- Se você extraiu para **Downloads**, pode arrastar a pasta **DoZero** para **Área de Trabalho** ou **Documentos** — escolha onde prefere manter o projeto.

6. Abrir a pasta no VS Code

- O passo para abrir a pasta no VS Code é **igual** ao que você já aprendeu para a pasta criada anteriormente.
- Agora a pasta **DoZero** está aberta no VS Code e você pode seguir com o projeto normalmente.

Parte 6

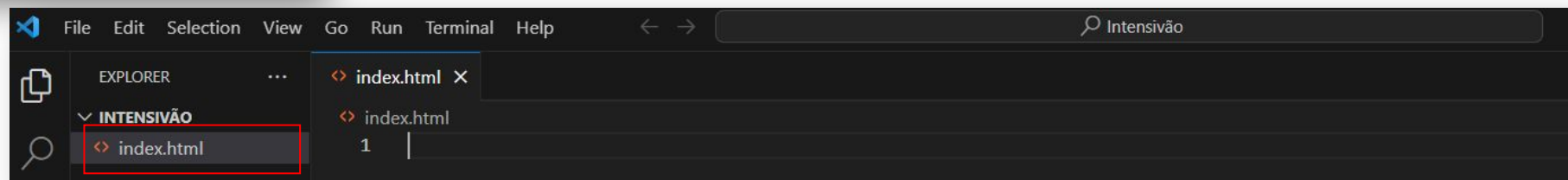
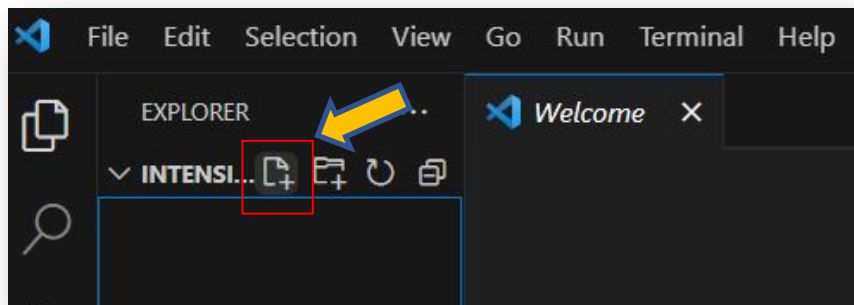
Estruturas – HTML/CSS

Estruturas – HTML / CSS

O arquivo "index.html" é um arquivo HTML que serve como a página inicial do seu site. Ele contém a estrutura básica do seu site, incluindo a marcação HTML, os estilos CSS e os scripts JavaScript. É a partir desse arquivo que você irá construir e desenvolver o conteúdo do seu site, adicionando elementos, estilos e funcionalidades.

Ao abrir o seu site no navegador, o arquivo "index.html" será o primeiro a ser carregado e exibido como a página inicial do seu site. É nele que você irá definir o layout, a estrutura e o conteúdo principal do seu site.

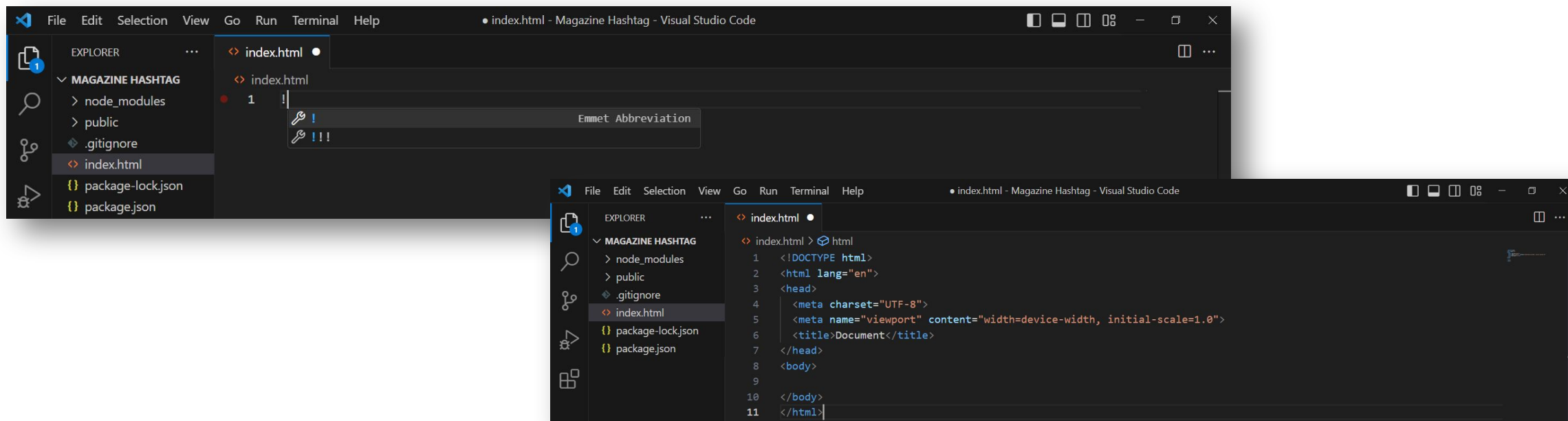
Vamos agora criar o arquivo "index.html". Para isso, clique no ícone "Novo Arquivo", escreva "index.html" e pronto! Dessa forma, você terá criado um novo arquivo.



Estruturas – HTML / CSS

Agora você pode começar a editar o arquivo "index.html" no VS Code, adicionando o código HTML necessário para construir o audiobook.

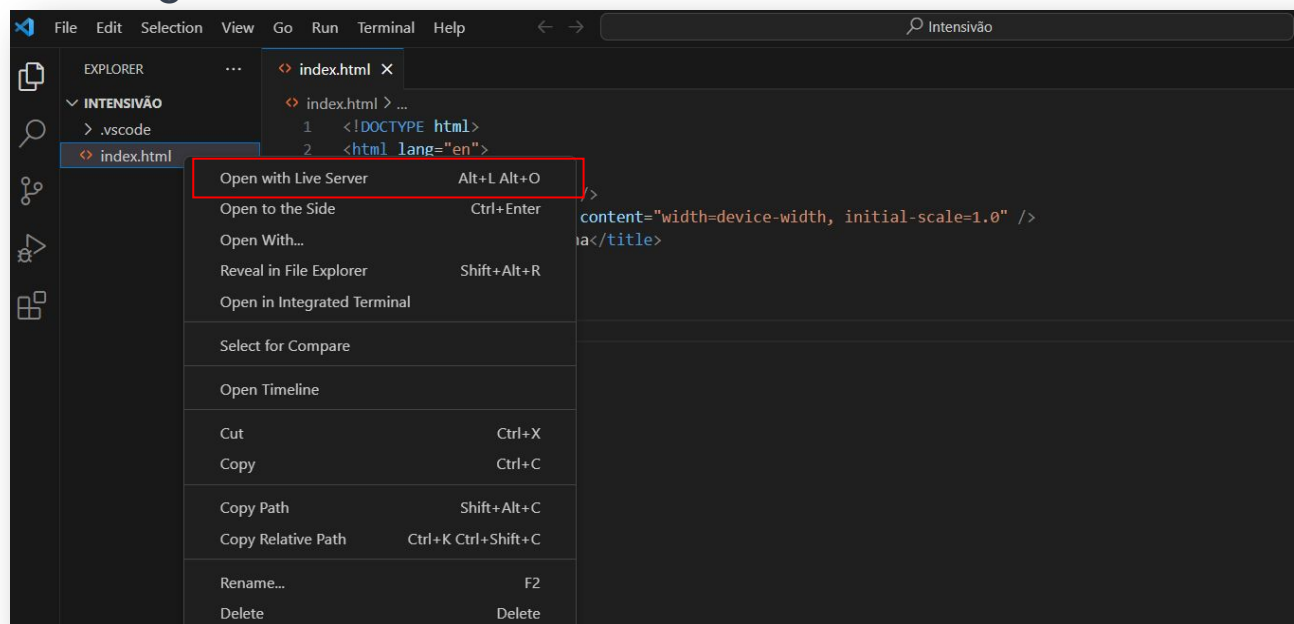
- Com o arquivo "index.html" aberto no VS Code, na linha em branco dentro do arquivo digite (!) + tab ou Enter.
- O VS Code irá expandir automaticamente o snippet ! para um modelo básico de arquivo HTML.
- O modelo básico de arquivo HTML incluirá a estrutura básica do HTML, incluindo as tags <html>, <head> e <body>.



Estruturas – HTML / CSS

Para visualizar a sua primeira página web, você utilizará a extensão Live Server, uma ferramenta incrivelmente útil que torna o desenvolvimento web mais dinâmico e eficiente. Para aplicá-la a um arquivo **index.html**, basta seguir estes passos:

- Com o arquivo **index.html** aberto, clique com o botão direito no editor.
- Selecione a opção "Open with Live Server" no menu suspenso.
- O Live Server iniciará automaticamente, e o seu navegador padrão será aberto, exibindo a página **index.html**.
- Agora, sempre que você fizer alterações no arquivo **index.html** e salvar, o Live Server atualizará automaticamente o navegador.



Estruturas - HTML / CSS

A estrutura básica do HTML gerada pelo VS Code inclui a declaração do tipo de documento (**<!DOCTYPE html>**), a **tag <html>** que envolve todo o conteúdo da página, a **tag <head>** que contém informações sobre a página, como o título exibido na aba do navegador (**tag <title>**), e a **tag <body>** onde você irá adicionar o conteúdo visível do seu site.

Agora você pode começar a adicionar o conteúdo do seu site dentro das **tags <body>...</body>**. Por exemplo, você pode adicionar cabeçalhos, parágrafos, imagens, links e outros elementos HTML.



Em HTML, uma **tag** é um elemento fundamental usado para estruturar e formatar o conteúdo de uma página da web. As **tags** são escritas **entre os símbolos < e >**, e podem conter atributos e/ou texto.

As **tags HTML** são essenciais para estruturar e organizar o conteúdo de uma página da web. Elas permitem que você crie seções, formate o texto, insira imagens, crie links e muito mais.

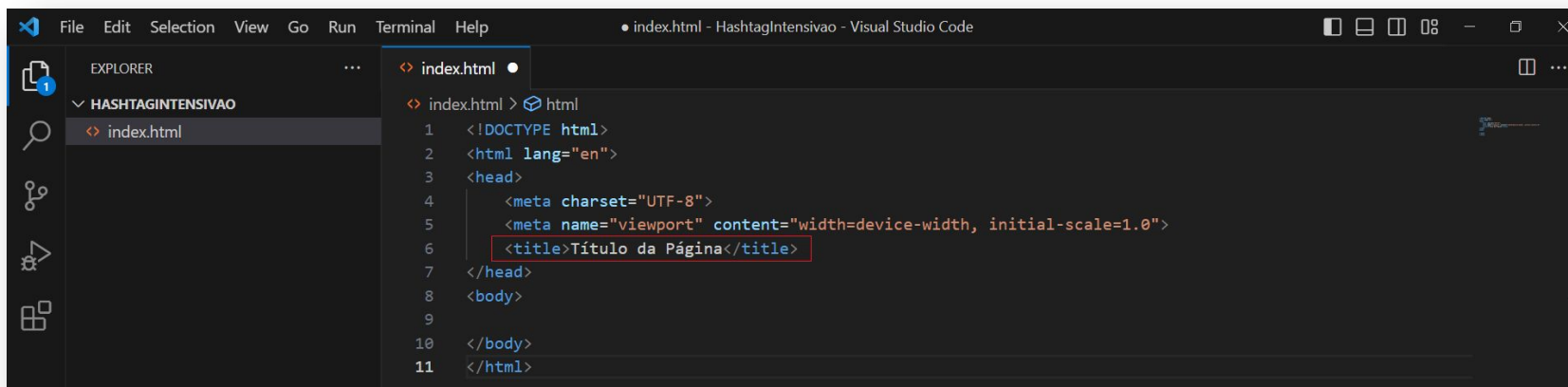
Ao escrever código HTML, é importante usar as **tags** corretas e fechá-las adequadamente para garantir que a página seja exibida corretamente nos navegadores.

Compreender e utilizar as **tags** corretamente é fundamental para criar páginas da web bem estruturadas e funcionais.

Estruturas – HTML / CSS

A **tag <title>** é usada dentro da **seção <head>** do seu documento HTML para definir o título da página. O título é exibido na barra de título do navegador e também pode ser exibido em outros lugares, como nos resultados de pesquisa.

É importante notar que o título da página deve ser relevante e descritivo para que os usuários possam entender o conteúdo da página antes mesmo de abri-la. Além disso, os mecanismos de busca também levam em consideração o título da página ao exibir os resultados de pesquisa.



```
File Edit Selection View Go Run Terminal Help
index.html - HashtagIntensivao - Visual Studio Code

EXPLORER
HASHTAGINTENSIVAO
index.html

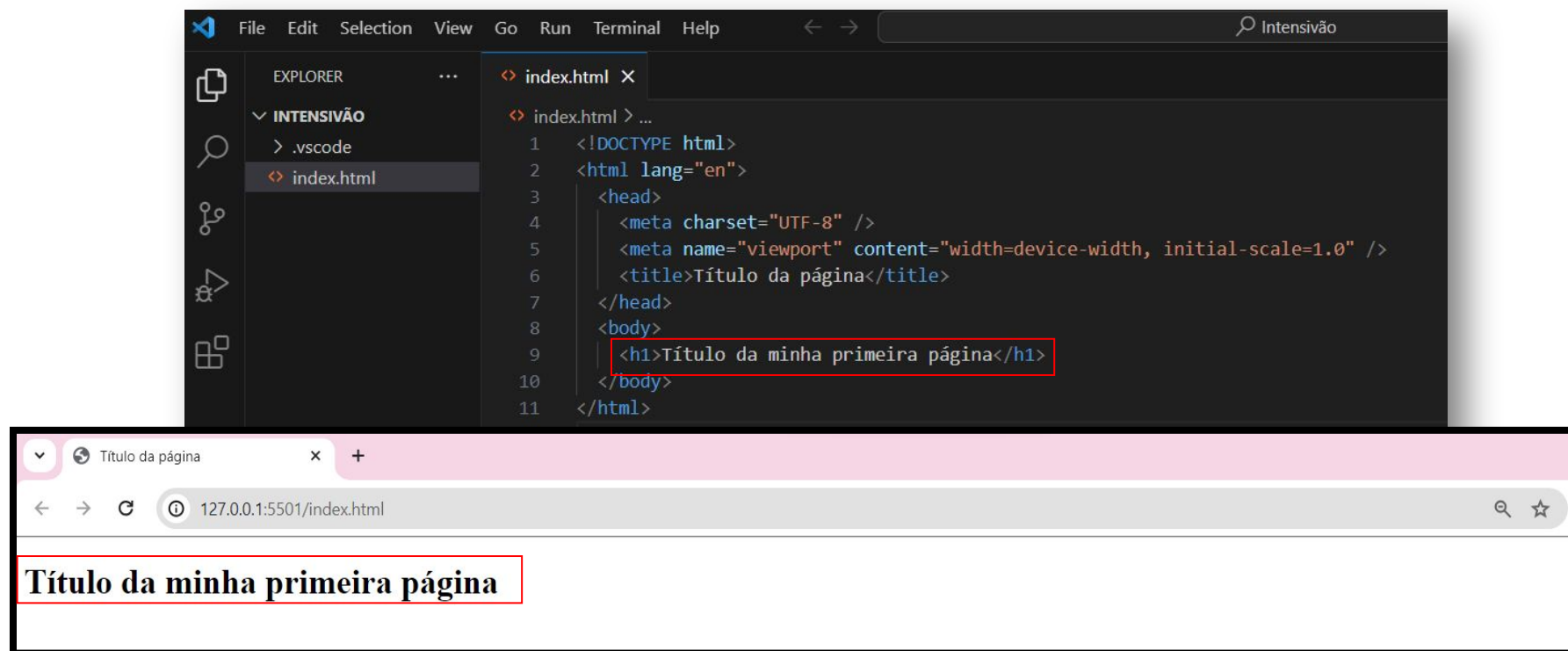
index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Título da Página</title>
7 </head>
8 <body>
9
10 </body>
11 </html>
```



Estruturas – HTML / CSS

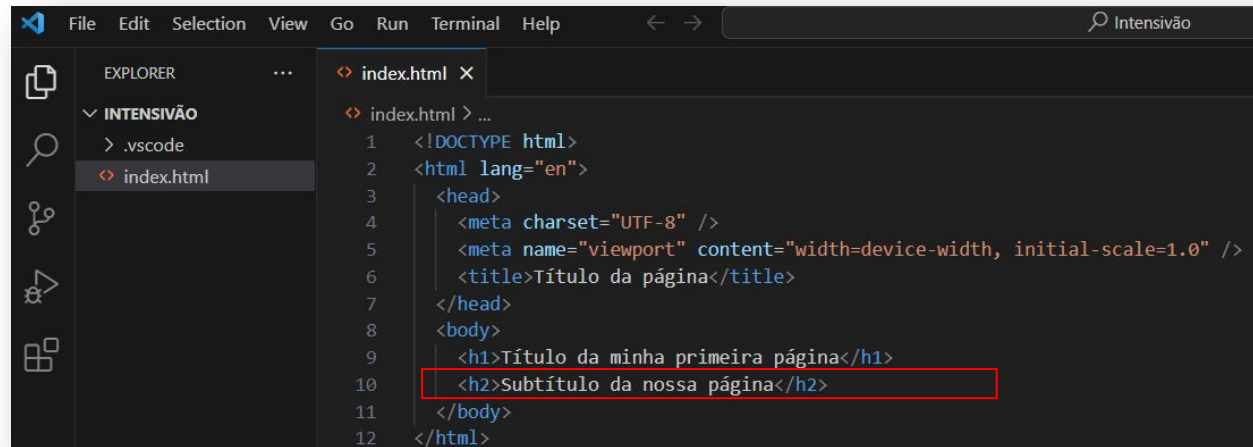
A tag **<h1>** é uma tag HTML que representa o título principal de uma seção ou página em um documento. O "h" em **<h1>** indica "header" (cabeçalho), e o número "1" indica o nível de importância do título. Portanto, **<h1>** é usado para o título mais importante e geralmente é o primeiro título em uma hierarquia de títulos HTML.

O objetivo principal da tag **<h1>** é destacar visualmente o texto dentro dela como o título mais significativo. O texto envolto pela tag **<h1>** geralmente é exibido com uma formatação maior e mais audaciosa do que o restante do conteúdo na página.

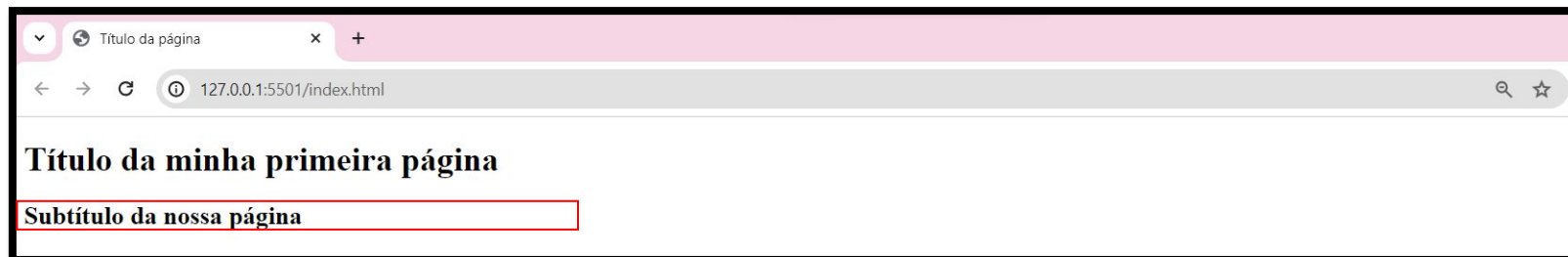


Estruturas – HTML / CSS

Além do **<h1>**, há também tags **<h2>**, **<h3>**, **<h4>**, **<h5>**, e **<h6>**, cada uma representando títulos de importância decrescente. Essa hierarquia é valiosa para estruturar a página e indicar a relação entre diferentes seções.



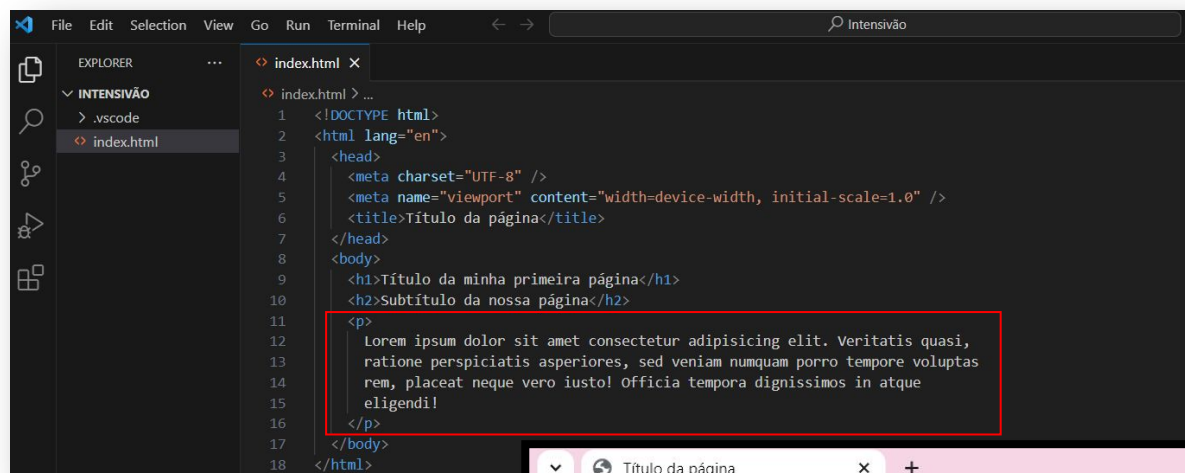
```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Título da página</title>
7   </head>
8   <body>
9     <h1>Título da minha primeira página</h1>
10    <h2>Subtítulo da nossa página</h2>
11  </body>
12 </html>
```



Estruturas – HTML / CSS

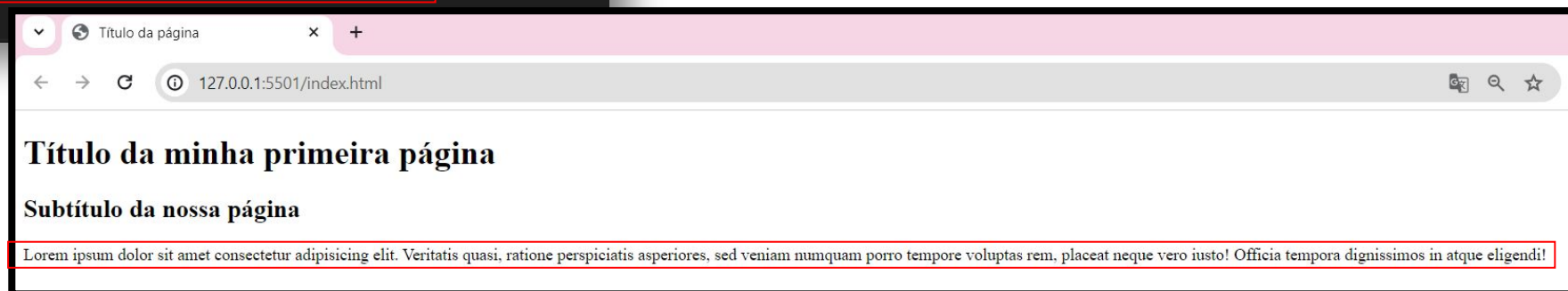
A **tag <p>** é uma tag HTML usada para definir um parágrafo de texto em um documento HTML. A **tag <p>** é usada para separar e estruturar o conteúdo de texto em parágrafos distintos.

A **tag <p>** é útil para organizar e estruturar o conteúdo de texto em um documento HTML. Ela também é importante para a acessibilidade, pois ajuda a tornar o conteúdo mais legível e compreensível para os usuários



```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Título da página</title>
7   </head>
8   <body>
9     <h1>Título da minha primeira página</h1>
10    <h2>Subtítulo da nossa página</h2>
11    <p>
12      Lorem ipsum dolor sit amet consectetur adipisicing elit. Veritatis quasi,
13      ratione perspiciatis asperiores, sed veniam numquam porro tempore voluptas
14      rem, placeat neque vero iusto! Officia tempora dignissimos in atque
15      eligendi!
16    </p>
17  </body>
18 </html>
```

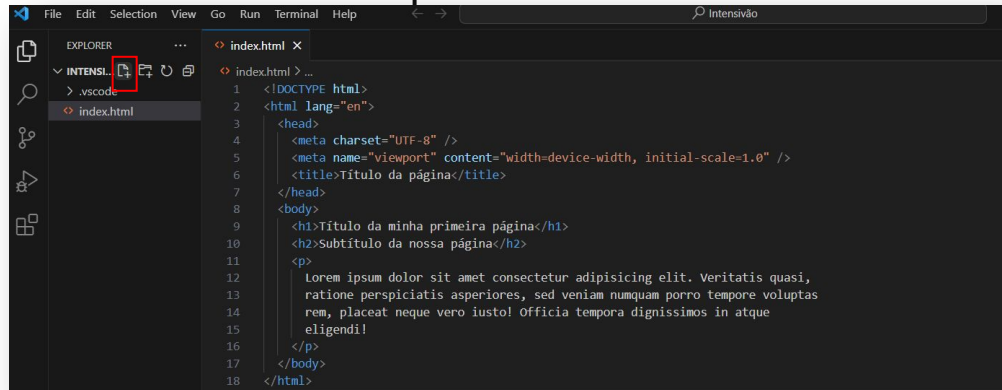
Se você escrever "lorem" dentro de uma tag <p>, estará utilizando um texto de preenchimento padrão conhecido como "Lorem Ipsum". Esse texto é frequentemente utilizado na indústria de design e tipografia para preencher espaços temporários e visualizar o layout de uma página antes que o conteúdo real seja inserido. O "Lorem Ipsum" não possui significado real em nenhum idioma conhecido, mas suas palavras e estrutura de frase se assemelham a um texto legível, proporcionando uma representação visual do layout do conteúdo.



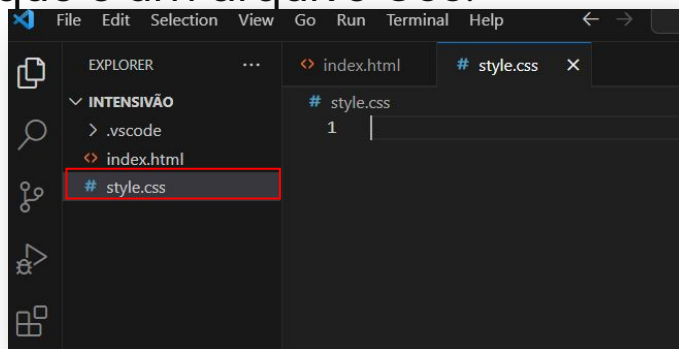
Estruturas – HTML / CSS

Nesse momento iremos criar nosso arquivo **style.css**. O arquivo style.css é um arquivo que contém as regras de **estilo em CSS** (Cascading Style Sheets) para um documento HTML. Ele é usado para adicionar estilos e personalizar a aparência dos elementos HTML em uma página da web.

No VS Code clique no ícone adicionar novo arquivo:



- Crie um **arquivo style.css** na mesma pasta onde o documento HTML está localizado. Certifique-se de usar a extensão .css para indicar que é um arquivo CSS.



Estruturas – HTML / CSS

Você pode adicionar as regras de estilo no arquivo style.css para estilizar os elementos HTML. Por exemplo, para definir a cor do texto do elemento de título (**h1**) como azul e o subtítulo (h2) como vermelho, você pode adicionar a seguinte regra CSS no arquivo style.css:

```
# style.css > ...
1  h1 {
2    color: blue;
3  }
4
5  h2 {
6    color: red;
7  }
```

No documento HTML, adicione a seguinte linha dentro da **tag <head>** para vincular o arquivo style.css ao documento HTML:

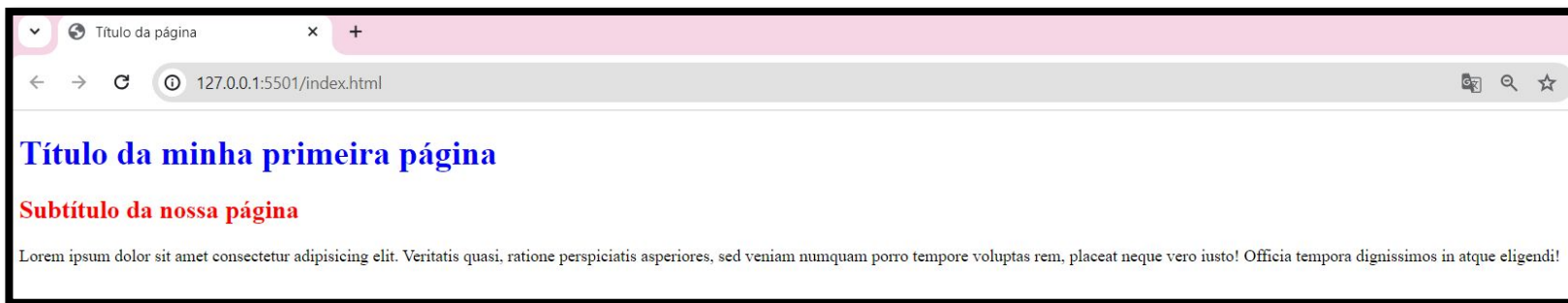
```
<? index.html • # style.css
<? index.html > html > head
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Título da página</title>
7    link
8  </head> link
9  <body> link:atom
10 <p> link:css Emmet Abbreviation
11 <di> link:favicon
12 </body> link:im
13 </html> link:import
```

```
<? index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6    <title>Título da página</title>
7    <link rel="stylesheet" href="./style.css" />
8  </head>
9  <body>
10 <h1>Título da minha primeira página</h1>
11 <h2>Subtítulo da nossa página</h2>
12 <p>
13   Lorem ipsum dolor sit amet consectetur adipisicing elit. Veritatis quasi,
14   ratione perspiciatis asperiores, sed veniam numquam porro tempore voluptas
15   rem, placeat neque vero iusto! Officia tempora dignissimos in atque
16   eligendi!
17 </p>
18 </body>
19 </html>
```

Estruturas – HTML / CSS

Essa linha de código adiciona um link para o arquivo style.css na página HTML, permitindo que as regras de estilo sejam aplicadas. O **href="/style.css"** é um atributo da **tag <link>** que especifica o local do arquivo CSS que será vinculado ao documento HTML.

Depois de adicionar as regras de estilo no arquivo style.css, salve-o. As regras de estilo serão aplicadas automaticamente aos elementos HTML correspondentes quando a página for carregada no navegador.

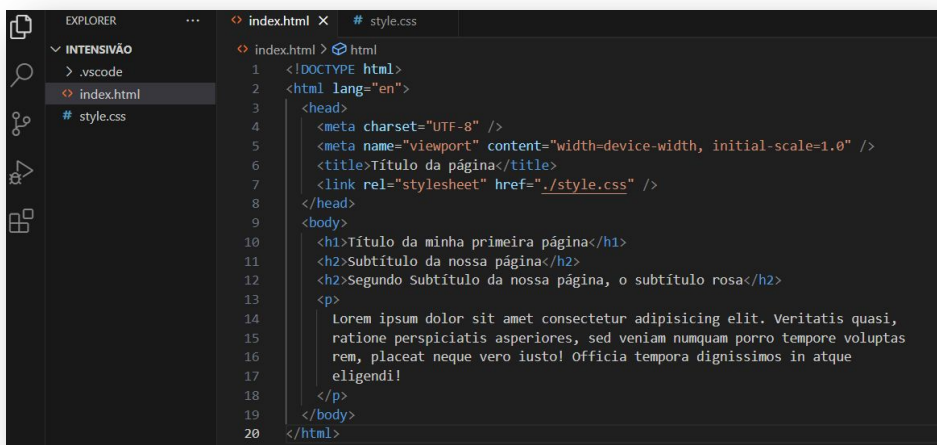


Ao criar conteúdo em HTML, você pode usar tags diversas vezes para representar diferentes elementos na sua página web. Essa prática é fundamental para estruturar e organizar o conteúdo de maneira clara e semântica. Cada tag, como **<h1>**, **<h2>**, **<p>**, entre outras, desempenha um papel específico na definição do tipo de conteúdo e na formatação visual.

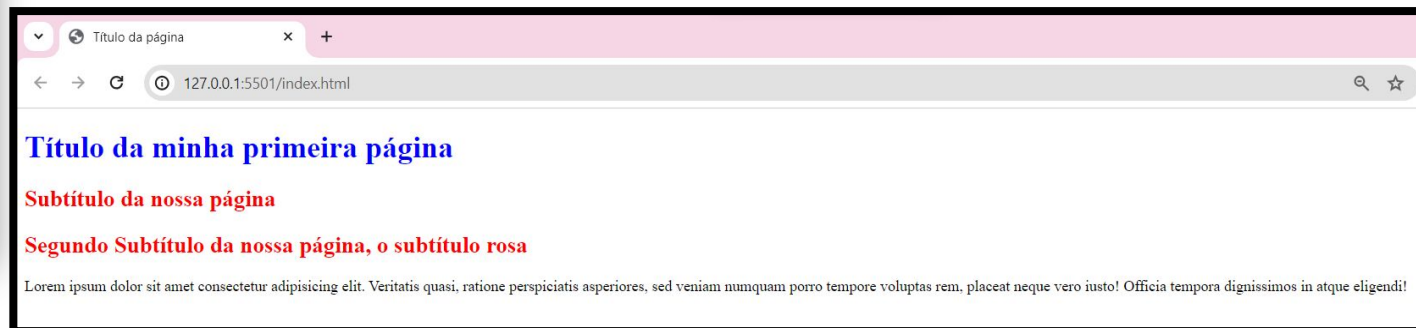
Estruturas – HTML / CSS

Por exemplo, ao usar a tag **<h1>** repetidamente, você pode criar vários títulos de diferentes seções. Cada instância da tag **<p>** pode representar parágrafos distintos. O mesmo princípio se aplica a diversas outras tags HTML, permitindo a criação de uma estrutura lógica e bem organizada para o seu conteúdo.

Ao utilizar tags múltiplas, você molda a aparência e a funcionalidade da sua página, proporcionando aos usuários uma experiência de navegação consistente e compreensível. Essa abordagem é essencial para o desenvolvimento web eficaz, pois permite a representação clara e estruturada de informações variadas em um ambiente online.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Título da página</title>
7   <link rel="stylesheet" href="./style.css" />
8 </head>
9 <body>
10  <h1>Título da minha primeira página</h1>
11  <h2>Subtítulo da nossa página</h2>
12  <h2>Segundo Subtítulo da nossa página, o subtítulo rosa</h2>
13  <p>
14    Lorem ipsum dolor sit amet consectetur adipisicing elit. Veritatis quasi,
15    ratione perspiciatis asperiores, sed veniam numquam porro tempore voluptas
16    rem, placeat neque vero iusto! Officia tempora dignissimos in atque
17    eligendi!
18  </p>
19 </body>
20 </html>
```



Estruturas – HTML / CSS

É importante lembrar que você pode adicionar várias regras de estilo no arquivo `style.css` para estilizar diferentes elementos HTML. Você pode utilizar **seletores CSS** para segmentar elementos específicos, como **classes**, **IDs**, **tags HTML**, etc., e aplicar estilos personalizados a esses elementos.

Além disso, você pode utilizar várias propriedades CSS para definir diferentes estilos, como **cor de texto**, **cor de fundo**, **tamanho da fonte**, **margens**, **preenchimento**, **alinhamento**, **bordas**, entre muitos outros.

O atributo `class` em HTML é utilizado para designar **uma ou mais classes** a um elemento específico. Classes são identificadores reutilizáveis que podem ser estilizados no CSS, permitindo a aplicação consistente de estilos a diferentes partes da sua página.

Ao atribuir uma classe a um elemento HTML, você está essencialmente dizendo que este elemento faz parte de um grupo específico. Por exemplo, você pode ter uma classe chamada `"destaque"` que define um estilo de fundo amarelo para todos os elementos que compartilham essa classe.

No CSS, você define como essas classes devem ser estilizadas. Ao invés de aplicar estilos diretamente a uma tag HTML específica (como `<p>` ou `<div>`), você pode criar classes no CSS e aplicá-las a qualquer elemento que precise desses estilos. Isso torna o código mais modular e fácil de manter, já que você pode ajustar os estilos de toda uma classe alterando apenas uma definição no CSS.

Além das classes, existe também o atributo `id`, que serve para **identificar um único elemento** de forma única dentro da página. Diferente da classe, que pode ser usada em vários elementos, o `id` deve ser **exclusivo** — ou seja, usado apenas uma vez por página. Ele é útil quando você quer aplicar um estilo ou comportamento específico a apenas um elemento. No CSS, os `ids` são representados com o símbolo `#`.

Estruturas – HTML / CSS

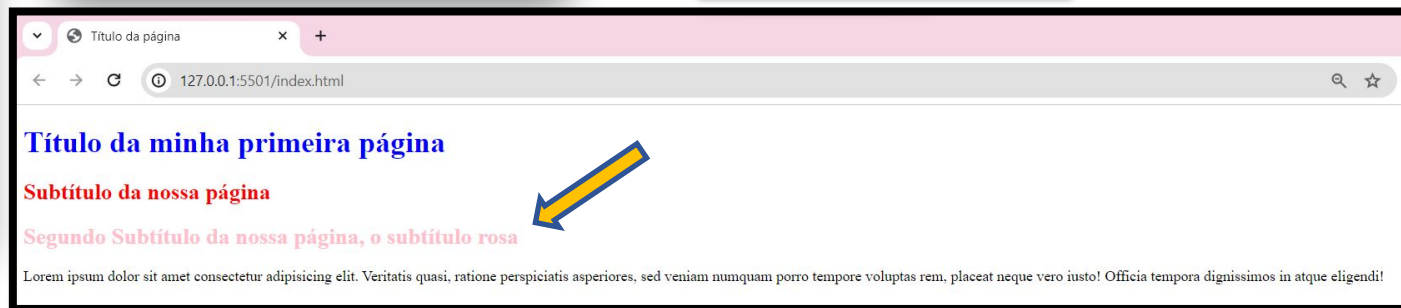
Vamos criar uma classe para o segundo elemento **<h2>**, chamada "texto-rosa". Para aplicarmos um estilo diferente do primeiro **<h2>**, iremos utilizar o arquivo "style.css" e atribuir a propriedade **color** ao seletor de classe ".texto-rosa".

Em outras palavras, queremos dar um estilo especial ao segundo **<h2>** chamando-o de "texto-rosa". Vamos definir as características visuais específicas para essa classe no arquivo de estilo, diferenciando-a do primeiro **<h2>**.

```
index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Título da página</title>
7     <link rel="stylesheet" href="./style.css" />
8   </head>
9   <body>
10    <h1>Título da minha primeira página</h1>
11    <h2>Subtítulo da nossa página</h2>
12    <h2 class="texto-rosa">
13      Segundo Subtítulo da nossa página, o subtítulo rosa
14    </h2>
15    <p>
16      Lorem ipsum dolor sit amet consectetur adipisicing elit. Veritatis quasi,
17      ratione perspiciatis asperiores, sed veniam numquam porro tempore voluptas
18      rem, placeat neque vero iusto! Officia tempora dignissimos in atque
19      eligendi!
20    </p>
21  </body>
22 </html>
```

```
# style.css > ...
1 h1 {
2   color: blue;
3 }
4
5 h2 {
6   <element class="texto-rosa">
7   Selector Specificity: (0, 1, 0)
8   .texto-rosa {
9     color: pink;
10  }
11 }
```

```
# style.css > ...
1 h1 {
2   color: blue;
3 }
4
5 h2 {
6   color: red;
7 }
8
9 .texto-rosa {
10  color: pink;
11 }
```

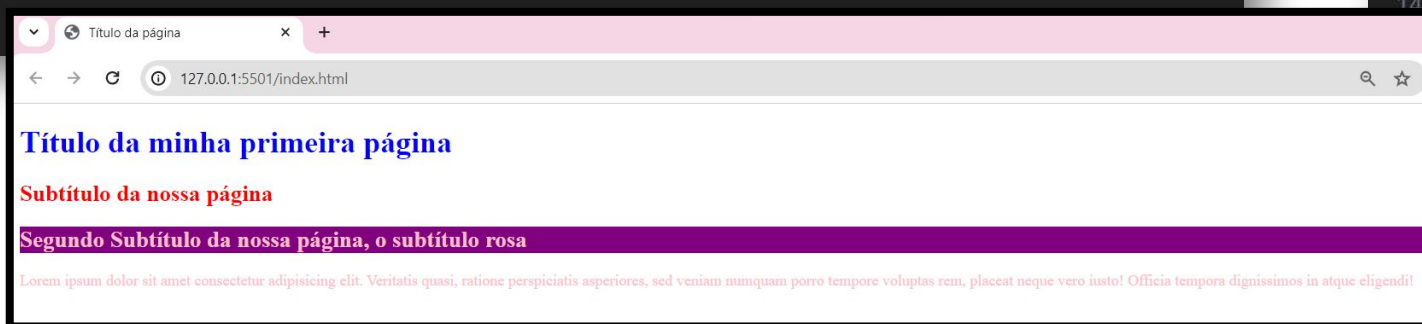


Estruturas – HTML / CSS

id e **class** são atributos em HTML para identificação e agrupamento de elementos, permitindo estilização e manipulação via CSS e JavaScript. **id** é exclusivo por página, identificando singularmente um elemento, enquanto **class** é reutilizável, aplicada a vários elementos similares, promovendo uma abordagem modular no desenvolvimento web. Use **id** para singularidade e **class** para reusabilidade. No CSS, **id** é referenciado por (**#**), e **class** por (**.**).

```
<h1 id="titulo">Título da minha primeira página</h1>
<h2 id="subtitulo-1">Subtítulo da nossa página</h2>
<h2 id="subtitulo-2" class="texto-rosa">
  Segundo Subtítulo da nossa página, o subtítulo rosa
</h2>
<p class="texto-rosa">
  Lorem ipsum dolor sit amet consectetur adipisicing elit. Veritatis quasi,
  ratione perspiciatis asperiores, sed veniam numquam porro tempore voluptas
  rem, placeat neque vero iusto! Officia tempora dignissimos in atque
  eligendi!
</p>
```

```
# style.css > ...
1  h1 {
2    color: blue;
3  }
4
5  h2 {
6    color: red;
7  }
8
9  .texto-rosa {
10   color: pink;
11 }
12
13 #subtitulo-2 {
14   background-color: purple;
15 }
```



Parte 7

Iniciando Projeto - HTML

Iniciando Projeto - HTML

Agora que instalamos e tivemos o primeiro contato com o VS Code, compreendemos as estruturas HTML e CSS. Vamos iniciar nosso projeto Audiobook e o primeiro passo é baixar a pasta "DoZero" que irá conter as pastas "audios" e "imagens".

Esses arquivos são essenciais para a construção do nosso Audiobook, contendo os áudios dos capítulos do livro "Dom Casmurro" e a imagem do livro..

Após o download, você pode optar por 2 passos.

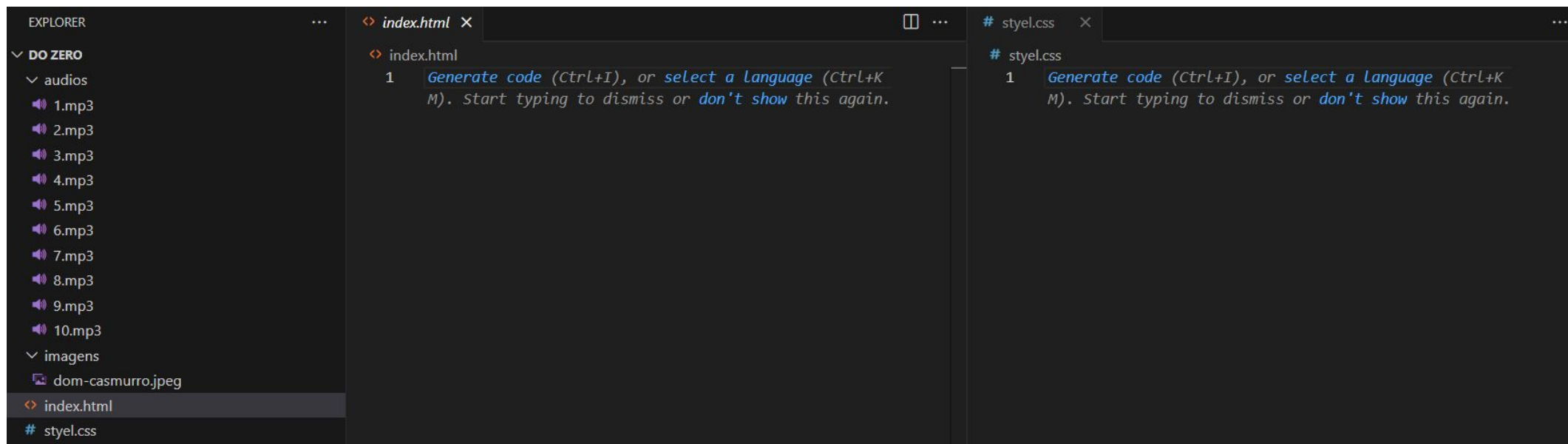
1- Adicione esses arquivos à pasta previamente criada chamada "Intensivao", então você irá recortar e colar as pastas "audios" e "imagens" para dentro do projeto..Remova o arquivo "style.css" que foi criado anteriormente (usado para entender a estrutura de um arquivo CSS) e apague o conteúdo do arquivo "index.html", pois iniciaremos novamente a sua estrutura.

2- Faça o download da pasta "DoZero", descompacte e abra o Vs Code. Vamos criar neste momento os arquivos index.html e o style.css dentro do projeto.



Iniciando Projeto - HTML

Neste ponto, a estrutura do seu projeto deve consistir em um arquivo "index.html" vazio, o arquivo "style.css" vazio, e as pastas "audios" e "imagens" localizadas dentro da pasta "Intensivão".



Iniciando Projeto - HTML

Dentro do arquivo "index.html", vamos começar a estrutura básica do HTML usando o snippet `!` + "Enter". Em seguida, vamos adicionar a **tag `<title>`**, que é usada para definir o **título da página** que aparece na aba do navegador. Vamos alterar o conteúdo desta tag para: "Hashtag Audiobooks". Logo depois, adicionamos o link para o nosso arquivo de estilos CSS. Para isso, usamos o snippet `link:css`, que gera a tag `<link>` com os atributos necessários. É importante fornecer o caminho correto para o arquivo, que neste caso é. Nesse momento você terá a seguinte estrutura:

```
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Hashtag Audiobooks</title>
7    <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10
11 </body>
12 </html>
```

Com isso, já podemos acionar a extensão Live Server e verificar que, embora não haja nenhum conteúdo dentro da nossa página, já conseguimos visualizar no navegador o endereço localhost executando no navegador.



Iniciando Projeto - HTML

Adicionaremos as primeiras tags html dentro do corpo da nossa página (tag `<body>`) que será a **tag `<div>`** e dentro dela adicionaremos a **tag `<img>`**.

```
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Document</title>
7      <link rel="stylesheet" href="./style.css" />
8    </head>
9    <body>
10     <div>
11       <img src="" alt="">
12     </div>
13   </body>
14 </html>
```

A **tag `<div>`** é uma das tags mais conhecidas e utilizadas no HTML. Ela é um elemento de divisão que permite agrupar e organizar o conteúdo de uma página. A **`<div>`** não possui um significado especial, mas é amplamente utilizada para criar blocos de conteúdo e aplicar estilos CSS a esses blocos.

A sintaxe da **tag `<div>`** é simples: você abre a tag com **`<div>`** e fecha com **`</div>`**. Todo o conteúdo que estiver entre essas tags será considerado parte da divisão. Por padrão, a **`<div>`** é um elemento de bloco, o que significa que ela gera uma quebra de linha automática

Iniciando Projeto - HTML

A tag **** em HTML é usada para incorporar imagens em uma página web. Duas propriedades-chave são:

- **src (source):** Especifica o caminho ou URL da imagem. Pode ser um caminho relativo no seu projeto ou uma URL completa.
- **alt (alternative text):** Fornece um texto alternativo para a imagem. É exibido se a imagem não carregar ou para usuários com leitores de tela. Essencial para acessibilidade e SEO, descrevendo o conteúdo ou propósito da imagem.

```
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Hashtag Audiobooks</title>
7      <link rel="stylesheet" href="style.css" />
8    </head>
9    <body>
10     <div class="container-aplicativo">
11       
12     </div>
13   </body>
14 </html>
```

Iniciando Projeto - HTML

Estrutura do áudio e informações da faixa

Antes de criarmos os controles, precisamos de um **reprodutor de áudio** e de uma **área que mostre os detalhes da faixa atual**.

O elemento **<audio>**:

```
<audio id="audio-capitulo" src="/audios/1.mp3"></audio>
```

- A tag **<audio>** é responsável por carregar e tocar arquivos de áudio no navegador.
- O atributo **src** define o caminho do arquivo (neste caso, **1.mp3**, dentro da pasta **audios**).
- O **id="audio-capitulo"** permite que possamos manipular esse áudio com JavaScript (ex.: dar play, pause, mudar o capítulo).

Não usamos o atributo **controls** porque queremos criar **nossos próprios botões personalizados**.

Iniciando Projeto - HTML

Área de informações da faixa

- Essa `<div>` principal (`informacoes-faixa`) agrupa os dados da faixa atual.
- `capitulo` mostra o número ou título da parte que está sendo reproduzida.
- `nome-autor` mostra o autor ou narrador.

```
<div id="informacoes-faixa">
  <div id="capitulo">Capítulo 1</div>
  <div id="nome-autor">Machado de Assis</div>
</div>
```

Por que separar assim?

- Facilita a **estilização no CSS** (título maior, autor menor, por exemplo).
- Deixa mais simples **atualizar dinamicamente** no JavaScript quando o usuário mudar de capítulo.

Iniciando Projeto - HTML

Controles com botões e SVGs

Agora que já temos a estrutura do áudio e suas informações, vamos criar **os botões de controle** para navegar e controlar a reprodução.

O que é um SVG?

- **SVG (Scalable Vector Graphics)** é um formato de imagem vetorial baseado em XML.
- Diferente de imagens comuns (como JPG ou PNG), o SVG pode ser ampliado ou reduzido sem perder qualidade.
- Podemos mudar sua cor e tamanho facilmente via **CSS** ou **JavaScript**.

Estrutura dos botões

Criamos um container para agrupar todos os controles e dentro dele colocamos cada botão com seu ícone SVG.

O que cada botão faz

- **Anterior:** volta ao início do capítulo ou ao capítulo anterior.
- **Play/Pause:** inicia ou pausa o áudio do capítulo atual.
- **Próximo:** avança para o próximo capítulo.

Iniciando Projeto - HTML

Estrutura dos botões

1

```
<div id="container-botoes">
  <svg
    id="anterior"
    xmlns="http://www.w3.org/2000/svg"
    viewBox="0 0 24 24"
    fill="currentColor"
  >
    <path
      d="M9.195 18.44c1.25.714 2.805-.189 2.805-1.629v-2.3416.945
      3.968c1.25.715 2.805-.188 2.805-1.628V8.69c0-1.44-1.555-2.343-2
      .805-1.628L12 11.029v-2.34c0-1.44-1.555-2.343-2.805-1.628l-7.108
      4.061c-1.26.72-1.26 2.536 0 3.256l7.108 4.061z"
    />
  </svg>

  <div id="play-pause">
    <svg
      xmlns="http://www.w3.org/2000/svg"
      viewBox="0 0 24 24"
      fill="currentColor"
      class="play"
    >
      <path
        fill-rule="evenodd"
        d="M2.25 12c0-5.385 4.365-9.75 9.75-9.75s9.75 4.365 9.75-4.365
        9.75-9.75 9.75s2.25 17.385 2.25 12Zm14.024-.983a1.125 1.125 0 0 1 0
        1.966l-5.603 3.113A1.125 1.125 0 0 1 9 15.113V8.887c0-.857.921-1.4
        1.671-.983l5.603 3.113z"
        clip-rule="evenodd"
      />
    </svg>
  </div>
</div>
```

2

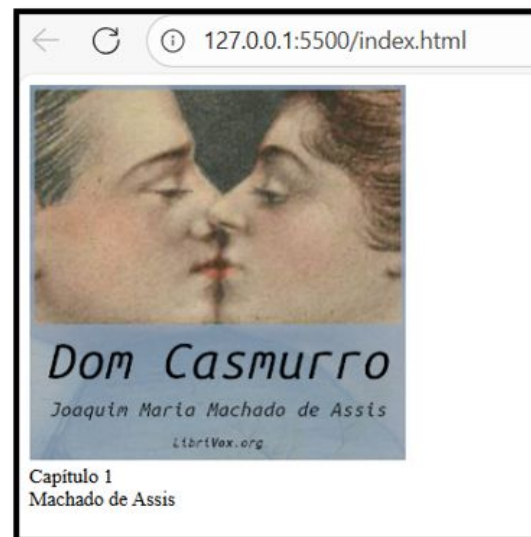
```
<svg
  xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 24 24"
  fill="currentColor"
  class="pause"
>
  <path
    fill-rule="evenodd"
    d="M2.25 12c0-5.385 4.365-9.75 9.75-9.75s9.75 4.365 9.75-4.365 9.75-9.75
    9.75s2.25 17.385 2.25 12Zm9 8.25a.75.75 0 0 0-.75.75v6c0 .414.336.75.75.75
    0 0 .75-.75V9a.75.75 0 0 0-.75-.75H9Zm5.25 0a.75.75 0 0 0-.75.75v6c0 .414.336.75.75
    .75H15a.75.75 0 0 0 .75-.75V9a.75.75 0 0 0-.75-.75h-.75z"
    clip-rule="evenodd"
  />
</svg>
</div>

<svg
  xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 24 24"
  fill="currentColor"
  id="proximo"
>
  <path
    d="M5.055 7.06C3.805 6.347 2.25 7.25 2.25 8.69v8.122c0 1.44 1.555 2.343 2.805 1.628L12
    14.471v2.34c0 1.44 1.555 2.343 2.805 1.628l7.108-4.061c1.26-.72 1.26-2.536 0-3.256l-7
    .108-4.061C13.555 6.346 12 7.249 12 8.689v2.34L5.055 7.061z"
  />
</svg>
</div>
```


Iniciando Projeto - HTML

Neste momento, após adicionarmos o **áudio** e as **informações do capítulo e autor**, o nosso projeto aparece assim na tela:

- Temos a **imagem da capa**.
- Logo abaixo, aparecem o **Capítulo** e o **Autor**.



Isso mostra que a estrutura básica em HTML está funcionando corretamente (como você vê na imagem exibida no navegador).

⚠ **Mas talvez você tenha notado que os botões (SVGs) ainda não aparecem.**

Isso acontece porque, até agora, não aplicamos nenhum **estilo com CSS** para organizar, posicionar e dar vida a esses elementos.

✅ **No próximo passo, vamos começar a estilizar nosso projeto** para que a tela fique mais parecida com um player de áudio real: com botões visíveis, organizados e bem apresentados.

Parte 8

Estilizando Projeto - CSS

Estilizando Projeto - CSS

Agora que já temos a **estrutura HTML** pronta, vamos começar a **estilizar nosso player de áudio**. O objetivo é deixar o projeto mais organizado, bonito e com a aparência de um aplicativo real.

1. Resetando os estilos padrões

Por padrão, cada navegador aplica alguns estilos automáticos (como espaçamentos). Aqui nós **zeramos a margem e o padding de todos os elementos**, e usamos `box-sizing: border-box` para facilitar o controle do tamanho dos elementos (as bordas e espaçamentos já são incluídos no cálculo de largura/altura).

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Estilizando Projeto - CSS

2. Estilo geral do `body`

- Usamos a fonte Montserrat para dar um visual moderno.
- Definimos um tamanho de fonte base `1.2rem`.
- O fundo recebe um degrade em tons de verde escuro, criando o clima de player.
- `min-height: 100vh` garante que ocupe 100% da altura da tela.
- O `display: flex` centraliza tudo vertical e horizontalmente.
- A cor do texto é branca para ter contraste com o fundo.

```
body {  
  font-family: "Montserrat", sans-serif;  
  font-size: 1.2rem;  
  background-image: linear-gradient(  
    to bottom,  
    hsl(141deg 75% 15%),  
    hsl(141deg 75% 5%)  
  );  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  flex-direction: column;  
  align-items: center;  
  color: #ffffff;  
}
```

Estilizando Projeto - CSS

3. Organização do container principal

- Esse container é responsável por **organizar todo o aplicativo** em coluna e manter tudo centralizado.

```
.container-aplicativo {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

4. Configuração dos botões

- Aqui estamos **removendo o estilo padrão dos botões** (cor, borda, fundo) e deixando apenas o que precisamos. Assim, os botões ficam prontos para receber os **ícones em SVG**.

```
button {  
  background-color: inherit;  
  color: inherit;  
  border: none;  
  cursor: pointer;  
}
```

5. Estrutura do player (música)

- Deixa a área do player organizada em coluna e centralizada.

```
.music-container {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

Estilizando Projeto - CSS

6. Informações da faixa

- As informações (capítulo e autor) ficam uma embaixo da outra e alinhadas no centro.
- O título do capítulo ganha **destaque em tamanho e negrito**.

```
#play-pause,
#anterior,
#proximo {
  cursor: pointer;
}

#anterior,
#proximo {
  width: 40px;
}

#play-pause {
  width: 60px;
}
```

```
#informacoes-faixa {
  display: flex;
  flex-direction: column;
  align-items: center;
  margin: 2rem 0 1rem 0;
}

#capitulo {
  font-size: 1.4rem;
  font-weight: bold;
}
```

7. Ajustando os botões (SVGs)

- O cursor muda para a **mãozinha de clique** quando passamos em cima dos botões.
- Definimos larguras diferentes: **play/pause maior (60px)** e **anterior/próximo menores (40px)**.

Estilizando Projeto - CSS

```
.pause {  
  display: none;  
}  
  
.tocando .play {  
  display: none;  
}  
  
.tocando .pause {  
  display: block;  
}
```

8. Controlando o estado do play/pause

Aqui está a mágica ✨:

- Por padrão, o ícone de **pause** fica escondido.
- Quando a música estiver tocando (classe **.tocando** aplicada no container via JS), o botão de **play** some e o de **pause** aparece.

9. Alinhando os botões

- Esse container organiza os três botões (**anterior, play/pause e próximo**) lado a lado, centralizados e com um espaço (**gap: 10px**) entre eles.

```
#container-botoes {  
  width: 100%;  
  display: flex;  
  justify-content: center;  
  gap: 10px;  
}
```

Estilizando Projeto - CSS

Com esse CSS, já conseguimos ver **um player estilizado e funcionalmente organizado**.



Parte 9

Integrando Inteligência: Aplicando Javascript ao Audiobook

Integrando Inteligência: Aplicando Javascript ao Audiobook

Ao seguir esses dois passos, você estará preparando a base para implementar funcionalidades inteligentes ao seu audiobook utilizando Javascript. Essa abordagem facilitará a interação do usuário com o audiobook, proporcionando uma experiência mais dinâmica e envolvente.

Passo 1: Criação do Arquivo Javascript:

- Para começar, crie um arquivo na raiz do seu projeto chamado **script.js**. Este arquivo será responsável por conter o código JavaScript que implementará as funcionalidades inteligentes para o audiobook.

Passo 2: Vínculo entre HTML e script.js:

- Para garantir que as funcionalidades do script.js se apliquem à estrutura do audiobook que construímos, é essencial criar uma ligação entre o arquivo HTML e o script.js. Certifique-se de incluir o seguinte código no final da tag <body> do seu arquivo HTML:

```
24 <script src="script.js"></script>  
25 </body>
```



A tag **<script>** é uma tag HTML usada para incorporar ou referenciar código Javascript em uma página web. Essa tag permite a execução de scripts do lado do cliente, proporcionando interatividade e dinamismo às páginas.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Para iniciarmos com a construção do nosso código Javascript precisamos entender sobre o porquê selecionar elementos HTML no JavaScript é crucial para tornar páginas web dinâmicas e interativas.

Essa prática permite a manipulação dinâmica do conteúdo, atualizações em tempo real, validação de formulários, gerenciamento de eventos, dinamismo na estilização, manipulação de conteúdo e integração com dados externos. Essa interação entre o JavaScript e os elementos HTML é fundamental para criar uma experiência de usuário mais envolvente e funcional, além de possibilitar o desenvolvimento de aplicativos web completos e responsivos.

Para realizarmos essa tarefa vamos utilizar a estrutura de código:

```
JS script.js  
1 document.getElementById("play-pause");
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

1. document:

O que é:

- Pense no **document** como uma representação do documento HTML da sua página. É como se fosse uma grande pasta que contém todas as informações sobre o que está na sua página, como textos, imagens, botões, etc.

Para que serve:

- Quando você quer fazer alguma coisa na sua página (como mudar um texto ou mostrar/ocultar algo), você precisa dizer ao computador onde encontrar essa coisa. É aí que entra o **document**. Ele ajuda você a dizer ao computador qual parte da sua página você quer mexer.

2. Por que usamos o ponto (.) – No Contexto do Documento (document):

- Quando você vê o ponto . depois do **document** (como em **document.getElementById()**), é como se estivesse dizendo: "Ei, **document**, use o seu poder de busca para encontrar algo!"

Selecionar Elementos:

- O ponto . é usado para selecionar elementos específicos no HTML usando métodos como **getElementById**, **getElementsByClassName**, ou **getElementsByTagName**.

Integrando Inteligência: Aplicando Javascript ao Audiobook

3. `.getElementById()`:

O que é:

- Imagine que seu documento HTML é uma cidade, e cada coisa na cidade tem um endereço único. O `.getElementById()` é como um mapa que ajuda você a encontrar uma coisa específica na cidade.

Para que serve:

- Quando você usa `.getElementById()`, você está pedindo ao computador para encontrar algo na sua página usando o ID único desse algo. Um ID é como um número da casa em uma rua. É único, o que significa que não há dois números iguais.

Em Javascript, as **aspas simples** (') e as **aspas duplas** (") são usadas para criar strings, que são sequências de caracteres. A escolha entre aspas simples e duplas geralmente é uma questão de preferência do desenvolvedor, mas existem algumas considerações práticas.

O **ponto e vírgula** (;) é usado como um delimitador de instrução. Ele indica o final de uma instrução ou expressão.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Para finalizarmos, vamos armazenar o resultado de **document.getElementById("play-pause")** em uma variável chamada "botaoPlayPause", que será do tipo **const**.

Ao armazenar o elemento em uma variável, você evita que o JavaScript precise procurar o elemento no DOM toda vez que você quiser interagir com ele. Isso pode melhorar o desempenho, especialmente se você estiver acessando o mesmo elemento várias vezes em seu código.

```
JS script.js > ...  
1  const botaoPlayPause = document.getElementById("play-pause");
```

O que é uma Variável:

- Em programação, uma variável é um espaço de armazenamento nomeado que contém um valor. Elas são usadas para armazenar e manipular dados em um programa. Em JavaScript, você declara uma variável usando palavras-chave como **var**, **let**, ou **const**, seguidas pelo nome da variável.

Tipo const em JavaScript:

- **const** é uma palavra-chave em JavaScript usada para declarar uma variável cujo valor não pode ser reatribuído. Isso significa que, após atribuir um valor a uma variável usando **const**, você não pode mais alterar esse valor.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Agora vamos adicionar um id ao elemento <audio>, para conseguir armazená-lo dentro de uma variável.

```
<div class="container-aplicativo">
  
  <audio id="audio-capitulo" src="/audios/1.mp3"></audio>
```



E realizar o passo a passo novamente, para armazenar chamando o `document.getElementById("id_do_elemento")`.

```
JS script.js > ...
1  const botaoPlayPause = document.getElementById("play-pause");
2  const audio = document.getElementById("audio-capitulo");
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Em JavaScript, uma função é um bloco de código nomeado que pode ser definido e chamado para realizar uma tarefa específica. Funções ajudam a organizar o código, promovem a reutilização e tornam o código mais modular. Aqui estão os principais elementos de uma função em JavaScript:

```
function nomeDaFuncao(parametro1, parametro2, ...) {  
    // Código a ser executado  
    return resultado; // Opcional: a função pode retornar um valor  
}
```

- **function:** Palavra-chave que inicia a definição da função.
- **nomeDaFuncao:** Nome da função, que é utilizado para chamá-la posteriormente.
- **parametro1, parametro2, ...:** Parâmetros são valores que a função espera receber. Eles são opcionais.
- **{ ... }:** Corpo da função, onde o código a ser executado está contido.
- **return resultado;** A palavra-chave **return** é usada para retornar um valor da função. Isso é opcional e depende do propósito da função.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Vamos dar um passo importante na construção do nosso audiobook! Agora, vamos criar a primeira ação específica para o nosso projeto. O próximo passo consiste em desenvolver uma função que será responsável por iniciar a reprodução do áudio do capítulo.

```
function tocarFaixa() {  
  audio.play();  
}
```

Essa função tem um propósito simples: ela usa a propriedade `play()` de um elemento de áudio para iniciar a reprodução da faixa de áudio associada a esse elemento.

- `tocarFaixa()`: Este é o nome da função.
- `audio`: Este é o nome da variável que representa um elemento de áudio no código.
- `.play()`: Isso é chamado de método e é usado para começar a reprodução do áudio.

Portanto, quando esta função é chamada, ela basicamente inicia a reprodução da faixa de áudio associada ao elemento de áudio representado pela variável `audio`.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Para chamar ou executar uma função em JavaScript, você utiliza o nome da função seguido pelos parênteses (). Isso é conhecido como invocar a função.

```
// Chamada da função tocarFaixa()  
tocarFaixa();
```

Vamos utilizar uma ferramenta muito importante para o desenvolvedor, O DevTools, ou Ferramentas de Desenvolvedor, que é um conjunto de utilitários que são incorporados nos navegadores da web para ajudar os desenvolvedores a inspecionar, depurar e otimizar o código de uma página da web. Ele fornece várias ferramentas que facilitam o desenvolvimento, testes e diagnóstico de problemas no código.

Como Acionar o DevTools (Chrome):

1. Botão Direito + "Inspecionar" em qualquer lugar na página.
2. Ou pressione Ctrl + Shift + I (Windows/Linux) ou Cmd + Opt + I (Mac).
3. Ou pressione o atalho F12 ou Fn + F12.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Funcionalidades Principais do DevTools:

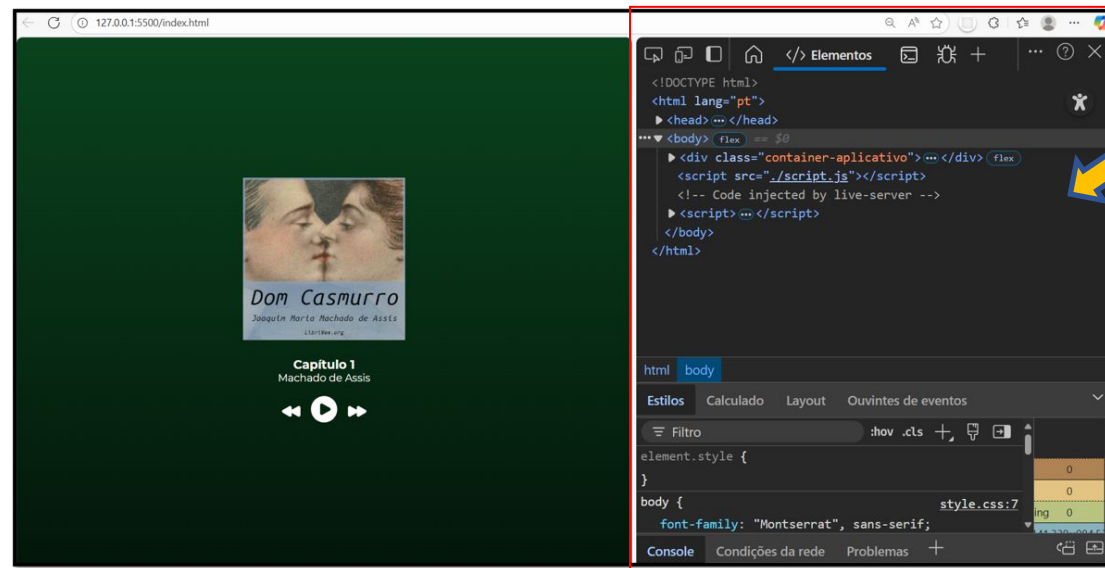
Elementos (ou Inspecionar): Permite examinar e modificar o HTML e o CSS de uma página interativamente.

- **Console:** Fornece um console interativo para a execução de comandos JavaScript, exibição de erros e mensagens de depuração.
- **Network (ou Rede):** Monitora as solicitações HTTP feitas pela página, permitindo análise de desempenho e diagnóstico de problemas de rede.
- **Sources (ou Origens):** Permite a edição, depuração e definição de pontos de interrupção no código JavaScript.
- **Application (ou Aplicação):** Oferece informações sobre armazenamento local, cookies, cache e outros recursos relacionados à aplicação.
- **Performance (ou Desempenho):** Ajuda a analisar e otimizar o desempenho da página.
- **Memory (ou Memória):** Monitora o uso de memória e identifica vazamentos de memória no código JavaScript.

Integrando Inteligência: Aplicando Javascript ao Audiobook

- **Security (ou Segurança):** Fornece informações sobre a segurança da conexão da página.
- **Audits (ou Auditorias):** Oferece auditorias automáticas para melhorar o desempenho, acessibilidade e as práticas de SEO da página.

Usar o DevTools é uma prática essencial para desenvolvedores web, pois proporciona uma visão detalhada e interativa do comportamento de uma página da web, ajudando a aprimorar a qualidade e o desempenho do código.



Integrando Inteligência: Aplicando Javascript ao Audiobook

A aba "Console" no DevTools é uma ferramenta poderosa e versátil que oferece várias funcionalidades úteis para os desenvolvedores. Aqui estão algumas razões pelas quais você pode querer utilizar a aba "Console":

Depuração de Código JavaScript:

- Permite a execução interativa de comandos JavaScript. Você pode testar expressões, verificar variáveis, e corrigir bugs diretamente no console.

•Exibição de Mensagens e Erros:

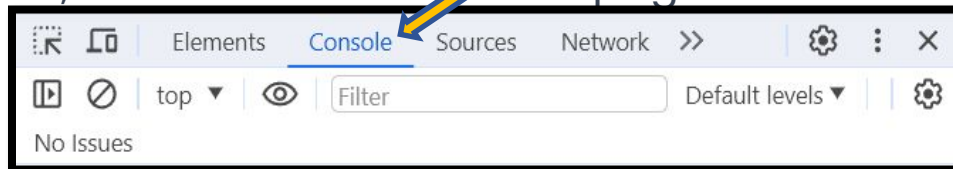
- Mostra mensagens de depuração, informações e erros gerados pelo seu código JavaScript. É uma ferramenta valiosa para identificar e corrigir problemas no código.

•Saída de Logs:

- Você pode usar **console.log()**, **console.warn()**, **console.error()**, entre outros, para exibir informações e mensagens de log diretamente no console. Isso é útil para entender o fluxo de execução do seu código.

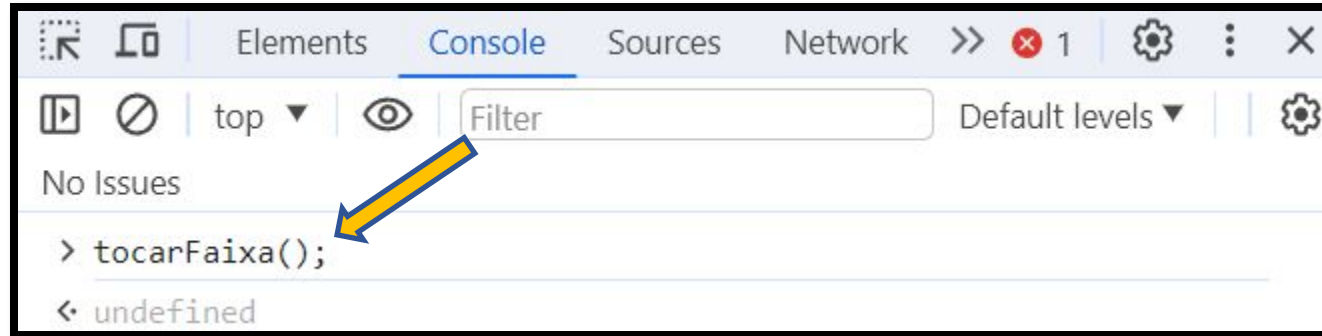
•Execução de Comandos Úteis:

- Além de JavaScript, você pode executar comandos úteis diretamente no console, como manipulações de DOM, acesso a elementos da página.



Integrando Inteligência: Aplicando Javascript ao Audiobook

Agora, faremos a chamada da função na aba "Console", pressionando ENTER. Vamos observar a mágica acontecer, com o programa reproduzindo o Capítulo 1 do nosso audiobook.



O próximo passo é adicionarmos um evento ao nosso botão para controlar a ação da nossa função "tocarFaixa()".

O que são Eventos:

- Eventos em JavaScript são ações ou ocorrências que acontecem no navegador, geralmente desencadeadas pelo usuário ou pelo próprio navegador. Eles são fundamentais para criar interatividade em páginas web.
- Eventos são como "coisas" que acontecem em uma página web. Elas podem ser ações do usuário (como clicar em um botão), interações do navegador (como a conclusão do carregamento de uma página), ou até mesmo mudanças no conteúdo da página.

Integrando Inteligência: Aplicando Javascript ao Audiobook

```
JS script.js > ...
1  const botaoPlayPause = document.getElementById("play-pause");
2  const audio = document.getElementById("audio-capitulo");
3
4  function tocarFaixa() {
5    |   audio.play();
6  }
7
8  botaoPlayPause.addEventListener("click", tocarFaixa);
```

Essa linha de código está dizendo ao computador para observar o botão chamado "botaoPlayPause", e quando alguém clicar nele, execute a função **tocarFaixa**.

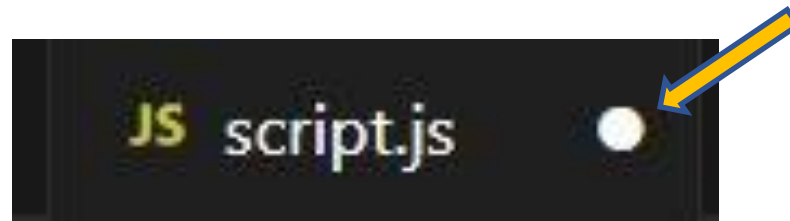
.addEventListener("click", tocarFaixa):

- Adiciona um "ouvinte" ao botão, esperando pela ação de clique ("**click**"). Em outras palavras, estamos dizendo ao computador para "escutar" quando alguém clica no botão.

Integrando Inteligência: Aplicando Javascript ao Audiobook

💡 Não se esqueça de que você pode explorar vários tipos de eventos na internet. Basta ir até um mecanismo de busca e procurar por eventos com JavaScript. Você encontrará sites e documentações que oferecem uma variedade de eventos para você ver e aprender, ampliando suas possibilidades de aplicação.

💡 Quando você faz alguma alteração no VS Code, os arquivos exibem um ícone de uma bolinha branca. Esse ícone indica que há alterações que ainda não foram salvas. Para salvar essas mudanças, você pode usar o atalho CTRL + S. Isso significa pressionar simultaneamente a tecla CTRL no seu teclado e, enquanto a mantém pressionada, pressionar a tecla S. Essa ação permite que as modificações que você fez sejam salvas, garantindo que o seu trabalho atualizado seja armazenado.



Integrando Inteligência: Aplicando Javascript ao Audiobook

Agora vamos criar a programação do nosso player de áudio.

1. Capturando os elementos do HTML

```
const nomeCapitulo = document.getElementById("capitulo");  
const audio = document.getElementById("audio-capitulo");  
const botaoPlayPause = document.getElementById("play-pause");  
const botaoProximoCapitulo = document.getElementById("proximo");  
const botaoCapituloAnterior = document.getElementById("anterior");
```

Aqui pegamos os elementos da nossa página (capítulo, áudio e botões) para que o JavaScript consiga manipulá-los.

2. Variáveis de controle

- **quantidadeCapitulos**: quantos áudios temos no projeto.
- **taTocando**: controla se o áudio está tocando ou pausado.
- **capitulo**: indica em qual capítulo estamos no momento (começando do 1).

```
const quantidadeCapitulos = 10;  
let taTocando = false;  
let capitulo = 1;
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Precisamos criar a lógica para que ocorra a troca dos ícones de play e pause, e vamos utilizar Javascript para isso utilizando os seguintes conceitos:

classList é uma propriedade de objetos DOM em JavaScript que fornece acesso às classes de um elemento HTML. Ela contém métodos para adicionar, remover e verificar a presença de classes em um elemento.

Métodos Principais:

.add("classe1", "classe2", ...): Adiciona uma ou mais classes ao elemento.

.remove("classe1", "classe2", ...): Remove uma ou mais classes do elemento.

3. Funções de tocar e pausar

- **tocarFaixa()** → inicia a reprodução e adiciona a classe **.tocando** para trocar o ícone do botão (play → pause).
- **pausarFaixa()** → pausa a música e remove a classe **.tocando**.

```
function tocarFaixa() {  
    audio.play();  
    taTocando = true;  
    botaoPlayPause.classList.add("tocando");  
}  
  
function pausarFaixa() {  
    audio.pause();  
    taTocando = false;  
    botaoPlayPause.classList.remove("tocando");  
}
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Ao explorarmos a aplicação das funções **tocarFaixa()** e **pausarFaixa()** em um elemento de botão, surge uma questão intrigante: como podemos designar funcionalidades a um único botão para desempenhar tanto a ação de reprodução quanto a de pausa em um áudio?

Essa dúvida é crucial, pois, à primeira vista, poderia parecer desafiador conciliar duas operações distintas em um único elemento. No entanto, é exatamente aqui que a flexibilidade e poder das estruturas de programação entram em cena.

Ao associarmos a função **tocarFaixa()** ao evento de clique em um botão, conseguimos iniciar a reprodução do áudio quando o botão é pressionado. Mas como podemos estender essa lógica para incluir a pausa? Este é um ponto intrigante, pois precisamos considerar como diferenciar entre as ações de tocar e pausar no mesmo botão.

Essa situação nos leva a explorar ainda mais a capacidade das estruturas condicionais em JavaScript. Podemos criar uma verificação que, ao ser acionada pelo clique no botão, avalia se o áudio está atualmente em reprodução ou pausado. Com base nessa avaliação, podemos então determinar se a função a ser executada será **tocarFaixa()** ou **pausarFaixa()**.

Integrando Inteligência: Aplicando Javascript ao Audiobook

ESTRUTURA CONDICIONAL:

A estrutura condicional é um conceito fundamental na programação, proporcionando a capacidade de tomar decisões dinâmicas com base em condições específicas. Em JavaScript, linguagem de programação amplamente utilizada para desenvolvimento web, as estruturas condicionais mais comuns são **if**, **else if** e **else**.

O bloco **if** é utilizado para executar um conjunto de instruções quando uma condição é avaliada como verdadeira. Por exemplo, podemos verificar se uma pessoa é maior de idade antes de permitir o acesso a determinadas funcionalidades.

Já o **else if** entra em cena quando há a necessidade de avaliar várias condições em sequência. Cada bloco **else if** é verificado apenas se as condições anteriores foram consideradas falsas. Isso é especialmente útil para situações em que diferentes ações precisam ser tomadas com base em diferentes cenários.

O bloco **else** é utilizado para definir um conjunto de instruções a ser executado quando nenhuma das condições anteriores for considerada verdadeira. Isso permite que um código mais abrangente seja desenvolvido, abordando todos os possíveis resultados.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Vamos criar uma função que terá o propósito de decidir se deve tocar ou pausar a faixa de áudio com base no estado atual.

function tocarOuPausarFaixa():

- Declaração de uma função chamada **tocarOuPausarFaixa**. Esta função será chamada quando o evento associado ao botão for acionado.

if (taTocando === true) { pausarFaixa(); } else { tocarFaixa(); }:

- Estrutura condicional (**if...else**) que verifica se a variável **taTocando** é igual a **true**.
- Se for verdadeiro (**taTocando === true**), significa que a faixa está sendo reproduzida, então a função **pausarFaixa()** é chamada para pausar a faixa.
- Se for falso (**else**), ou seja, a faixa não está sendo reproduzida, a função **tocarFaixa()** é chamada para começar a reprodução.

```
function tocarOuPausarFaixa() {  
  if (taTocando === true) {  
    pausarFaixa();  
  } else {  
    tocarFaixa();  
  }  
}
```

Essencialmente, essa função realiza uma ação de alternância entre tocar e pausar a faixa de áudio. Se a faixa estiver tocando, ela será pausada, e se estiver pausada, será tocada. A variável **taTocando** é usada como uma espécie de sinalizador para indicar o estado atual da reprodução da faixa. Essa abordagem é comum ao criar controles de reprodução toggle (alternância) em interfaces de áudio.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Vamos adicionar a declaração da variável `taTocando`, ela será do tipo "let" com o valor booleano de `false`.

let:

- A palavra-chave **let** é usada em JavaScript para declarar variáveis. Quando declaramos uma variável com **let**, estamos basicamente reservando um espaço na memória para armazenar informações que podem ser alteradas ao longo do tempo.

Booleano:

- É um tipo de dado em programação que pode ter apenas dois valores: **true** ou **false**. É frequentemente utilizado para representar estados binários, como ligado/desligado, verdadeiro/falso, ativo/inativo.

Agora, juntando esses conceitos, ao adicionar a declaração da variável **taTocando** com **let** e inicializá-la com o valor booleano **false**, estamos criando uma variável que pode ser usada para controlar o estado de reprodução da faixa de áudio. Por padrão, ela começa com o valor **false**, indicando que a faixa não está sendo tocada. Conforme o código evolui, esse valor pode ser atualizado para **true** quando a faixa estiver em reprodução e para **false** quando estiver pausada.

```
JS script.js > ...  
1  const botaoPlayPause = document.getElementById("play-pause");  
2  const audio = document.getElementById("audio-capitulo");  
3  let taTocando = false;
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Estamos usando a variável **taTocando** para controlar o estado da reprodução da faixa de áudio. Para fazer isso, dentro das funções **tocarFaixa()** e **pausarFaixa()**, iremos atualizar o valor dessa variável para refletir se a faixa está sendo tocada ou pausada.

tocarFaixa():

- Dentro desta função, vamos definir **taTocando** como **true**, indicando que a faixa está sendo tocada.

pausarFaixa():

- Dentro desta função, vamos definir **taTocando** como **false**, indicando que a faixa está sendo pausada.

Essas atualizações garantem que a variável **taTocando** reflète o estado atual da reprodução da faixa. Quando tocamos a faixa, **taTocando** se torna **true**, e quando pausamos, **taTocando** se torna **false**. Essa abordagem permite que o código saiba em que estado a faixa se encontra e tome decisões com base nesse estado.

```
function tocarFaixa() {  
  audio.play();  
  taTocando = true;  
  botaoPlayPause.classList.add("tocando");  
}  
  
function pausarFaixa() {  
  audio.pause();  
  taTocando = false;  
  botaoPlayPause.classList.remove("tocando");  
}
```


Integrando Inteligência: Aplicando Javascript ao Audiobook

Por fim, vamos utilizar a função "tocarOuPausarFaixa()" como parâmetro do método `addEventListener` que está associado ao elemento "botaoPlayPause".

```
botaoPlayPause.addEventListener("click", tocarOuPausarFaixa);
```



Integrando Inteligência: Aplicando Javascript ao Audiobook

Com a lógica de play e pause no lugar, o próximo passo será iniciar a narrativa no primeiro capítulo, e depois iremos explorar as funcionalidades de navegação, permitindo que os usuários retrocedam ou avancem entre os capítulos com facilidade. Essa adição não só incrementa a experiência de audição, mas também confere aos ouvintes o controle sobre sua própria jornada narrativa, capacitando-os a explorar e revisitando momentos específicos.

Vamos criar uma variável para representar os capítulos do nosso Audiobook. Como queremos que a jornada comece no capítulo um, vamos atribuir o valor 1 a essa variável.

```
1  const botaoPlayPause = document.getElementById("play-pause");
2  const audio = document.getElementById("audio-capitulo");
3  let taTocando = false;
4  let capitulo = 1;
```

E vamos adicionar também uma variável que armazenará o botão se avançar para o próximo capítulo:

```
JS script.js >  tocarFaixa
1  const botaoPlayPause = document.getElementById("play-pause");
2  const botaoProximoCapitulo = document.getElementById("proximo");
3  const audio = document.getElementById("audio-capitulo");
4  let taTocando = false;
5  let capitulo = 1;
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Vamos adicionar uma funcionalidade empolgante ao nosso audiobook chamada **proximoCapitulo()**. Ao acionar essa função, não só pulamos para o próximo capítulo na sequência, mas também nos certificamos de que o áudio correspondente a esse capítulo seja carregado e esteja pronto para tocar.

Isso significa que, ao navegar pelos capítulos da nossa história, o código automaticamente ajusta as configurações do áudio, proporcionando uma experiência fluida e contínua. Dessa forma, a função **proximoCapitulo()** não apenas avança a narrativa, mas também sincroniza o áudio para que os ouvintes possam aproveitar sem interrupções. É uma adição essencial para tornar a exploração dos capítulos do nosso audiobook ainda mais envolvente.

```
function proximoCapitulo() {  
  capítulo += 1;  
  audio.src = "/audios/" + capítulo + ".mp3";  
}
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Na função **proximoCapitulo()**, a primeira linha incrementa o número do capítulo (**capitulo**) em 1, avançando para o próximo na sequência. Na terceira linha, **audio.src** é atualizado para o novo caminho do arquivo de áudio correspondente ao próximo capítulo.

A linha **audio.src = "audios/dom-casmurro/" + capitulo + ".mp3";** é crucial para carregar o áudio correto. Aqui, **"audios/dom-casmurro/"** é o diretório onde os arquivos de áudio estão localizados, e **capitulo + ".mp3"** forma o nome do arquivo com base no número do capítulo. Essa concatenação dinâmica permite que o código se ajuste automaticamente ao capítulo atual, carregando o áudio apropriado.

Em resumo, ao acionar **proximoCapitulo()**, não apenas avançamos para o próximo capítulo, mas também garantimos que o áudio associado a esse capítulo seja carregado e preparado para ser reproduzido. Essa dinâmica facilita a navegação entre capítulos em nosso audiobook.

Agora, vamos conectar o botão de avançar para o próximo capítulo com o evento de clique.

```
botaoPlayPause.addEventListener("click", tocarOuPausarFaixa);  
botaoProximoCapitulo.addEventListener("click", proximoCapitulo);
```



Integrando Inteligência: Aplicando Javascript ao Audiobook

Aqui, foi introduzida uma estrutura condicional (if-else) para verificar se o incremento do capítulo ultrapassaria o limite máximo (**quantidadeCapitulos**). Se o capítulo atual for menor que o limite, incrementamos normalmente. Caso contrário, resetamos o capítulo para 1, garantindo que a sequência seja cíclica.

A utilização do if-else é interessante para manter a lógica da navegação entre capítulos consistente e controlada. Ele assegura que, ao chegar ao último capítulo, ao invés de continuar indefinidamente, a navegação retorne ao início da história.

A próxima funcionalidade que adicionaremos ao nosso projeto de Audiobook permitirá a navegação para o capítulo anterior. Vamos progredir na construção, seguindo a abordagem passo a passo que utilizamos na função **proximoCapitulo()**.

```
function proximoCapitulo() {
    pausarFaixa();

    if (capitulo < quantidadeCapitulos) {
        capitulo += 1;
    } else {
        capitulo = 1;
    }

    audio.src = "/audios/" + capitulo + ".mp3";
    nomeCapitulo.innerText = "Capítulo " + capitulo;
}
```

```
function capituloAnterior() {
    pausarFaixa();

    if (capitulo === 1) {
        capitulo = quantidadeCapitulos;
    } else {
        capitulo -= 1;
    }

    audio.src = "/audios/" + capitulo + ".mp3";
    nomeCapitulo.innerText = "Capítulo " + capitulo;
}
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Agora, é hora de ajustar a função **proximoCapitulo()**. Atualmente, não estamos controlando o número de faixas, ou seja, o projeto tem 10 faixas, representando 10 capítulos. Na implementação atual, não conseguimos parar no capítulo 10 e voltar para o capítulo 1; em vez disso, a navegação continuaria indefinidamente para 11, 12 e assim por diante, resultando em um erro quando não houver áudio correspondente.

Para evitar esse problema, precisamos adicionar um controle que verifique se atingimos o último capítulo. Se sim, ao acionar a função, ela deve nos levar de volta ao primeiro capítulo, criando uma experiência de navegação cíclica e evitando possíveis erros de áudio. Vamos realizar esse ajuste para garantir que nossa aplicação funcione de maneira mais suave e controlada.

Precisamos de uma variável que armazene o valor do número de capítulos.

```
const quantidadeCapitulos = 10;  
let taTocando = false;  
let capitulo = 1;
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Na função **capituloAnterior()**, a primeira linha decrementa o número do capítulo (**capitulo**) em 1, retrocedendo para o anterior na sequência.

Da mesma forma que fizemos na função **proximoCapitulo()**, será necessário aplicar uma lógica para controlar a quantidade de capítulos ao retroceder. Ao voltarmos do primeiro capítulo, a ideia é ir para o capítulo 10, que representa a última faixa do livro. Essa lógica garantirá uma navegação fluida, evitando erros e proporcionando uma experiência mais consistente ao ouvinte.

```
function capituloAnterior() {  
    pausarFaixa();  
  
    if (capitulo === 1) {  
        capitulo = quantidadeCapitulos;  
    } else {  
        capitulo -= 1;  
    }  
  
    audio.src = "/audios/" + capitulo + ".mp3";  
    nomeCapitulo.innerText = "Capítulo " + capitulo;  
}
```

Nessa estrutura condicional (if-else), verificamos se o capítulo é igual a 1. Se for verdadeiro (ou seja, estamos no primeiro capítulo), ajustamos o capítulo para ser igual à **quantidadeCapitulos** (representando o último capítulo do livro). Caso contrário (se não estivermos no primeiro capítulo), simplesmente decrementamos o capítulo em 1.

Essa lógica permite a navegação entre capítulos, indo do primeiro para o último e vice-versa.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Na função **capituloAnterior()**, a primeira linha decrementa o número do capítulo (**capitulo**) em 1, retrocedendo para o anterior na sequência.

Da mesma forma que fizemos na função **proximoCapitulo()**, será necessário aplicar uma lógica para controlar a quantidade de capítulos ao retroceder. Ao voltarmos do primeiro capítulo, a ideia é ir para o capítulo 10, que representa a última faixa do livro. Essa lógica garantirá uma navegação fluida, evitando erros e proporcionando uma experiência mais consistente ao ouvinte.

Agora, vamos adicionar uma variável que armazenará o botão de avançar para o próximo capítulo (**imagem 1**). Também associaremos o botão de retroceder para o capítulo anterior ao evento de clique (**imagem 2**).

```
1 const audio = document.getElementById("audio-capitulo");  
2 const botaoPlayPause = document.getElementById("play-pause");  
3 const botaoProximoCapitulo = document.getElementById("proximo");  
4 const botaoCapituloAnterior = document.getElementById("anterior");
```

1

```
botaoPlayPause.addEventListener("click", tocarOuPausarFaixa);  
botaoProximoCapitulo.addEventListener("click", proximoCapitulo);  
botaoCapituloAnterior.addEventListener("click", capituloAnterior);
```

2

Integrando Inteligência: Aplicando Javascript ao Audiobook

Chegamos aos ajustes finais do nosso projeto. Vamos incluir o nome do autor, o título do capítulo e a capacidade de alterar o nome dos capítulos. No arquivo **index.html**, encontramos os elementos que representarão esses dados no Audiobook.

```
<div class="container-aplicativo">
  

  <audio id="audio-capitulo" src="/audios/1.mp3"></audio>

  <div id="informacoes-faixa">
    <div id="capitulo">Capítulo 1</div>
    <div id="nome-autor">Machado de Assis</div>
  </div>
```


Integrando Inteligência: Aplicando Javascript ao Audiobook

Criaremos uma variável para armazenar o elemento responsável por guardar o nome do Capítulo.

```
const nomeCapitulo = document.getElementById("capitulo");
```

Vamos adicionar essa linha de código dentro das funções `proximoCapitulo()` e `capituloAnterior()`:

```
nomeCapitulo.innerText = "Capítulo " + capitulo;
```

Essa linha de código está associada à manipulação do conteúdo de um elemento HTML na página, mais especificamente, o elemento que armazena o título do capítulo. Aqui está uma explicação passo a passo: **nomeCapitulo**: Isso representa um elemento HTML que foi previamente selecionado no código, geralmente usando **document.getElementById()** ou métodos semelhantes. Esse elemento serve como o local onde desejamos exibir o título do capítulo.

.innerText: **innerText** é uma propriedade que permite modificar o texto interno de um elemento HTML. Ao usá-la, podemos alterar dinamicamente o conteúdo textual dentro do elemento.

"Capítulo " + capitulo: Aqui, estamos concatenando a string fixa "Capítulo " com a variável **capitulo**, que representa o número do capítulo atual. A concatenação (+) combina essas partes para criar uma nova string que representa o título formatado do capítulo.

Integrando Inteligência: Aplicando Javascript ao Audiobook

Nesse momento as funções `proximoCapitulo()` e `capituloAnterior()` terá essa estrutura:

```
function capituloAnterior() {
    pausarFaixa();

    if (capitulo === 1) {
        capitulo = quantidadeCapitulos;
    } else {
        capitulo -= 1;
    }

    audio.src = "/audios/" + capitulo + ".mp3";
    nomeCapitulo.innerText = "Capítulo " + capitulo;
}

function proximoCapitulo() {
    pausarFaixa();

    if (capitulo < quantidadeCapitulos) {
        capitulo += 1;
    } else {
        capitulo = 1;
    }

    audio.src = "/audios/" + capitulo + ".mp3";
    nomeCapitulo.innerText = "Capítulo " + capitulo;
}
```

Integrando Inteligência: Aplicando Javascript ao Audiobook

Agora precisamos criar uma lógica para quando a reprodução do áudio atual chegar ao fim, a função **proximoCapitulo** seja chamada, iniciando automaticamente a reprodução do próximo capítulo. Essa abordagem é útil para criar uma experiência contínua e automatizada ao ouvir um audiobook, onde a transição para o próximo capítulo ocorre sem a necessidade de intervenção do usuário.

```
botaoPlayPause.addEventListener("click", tocarOuPausarFaixa);  
botaoProximoCapitulo.addEventListener("click", proximoCapitulo);  
botaoCapituloAnterior.addEventListener("click", capituloAnterior);  
audio.addEventListener("ended", proximoCapitulo);
```

O trecho **audio.addEventListener('ended', proximoCapitulo);** refere-se a um evento que é acionado quando o áudio chega ao fim, ou seja, quando a reprodução da faixa de áudio atual é concluída.

- **'ended'**: é o tipo de evento que estamos ouvindo. Neste caso, estamos interessados no evento 'ended', que ocorre quando o áudio termina de ser reproduzido.

Integrando Inteligência: Aplicando Javascript ao Audiobook

🎉 **Parabéns!** 🎉

Você acaba de finalizar o projeto **Hashtag Audiobook!** 🚀

Ao longo dessa jornada, você:

- ✓ Criou a estrutura do seu projeto com **HTML**
- ✓ Estilizou e deu vida à interface com **CSS**
- ✓ Aprendeu a manipular o **DOM** com **JavaScript**
- ✓ Controlou áudios, botões, eventos e interações na página

Esse projeto mostrou, na prática, como HTML, CSS e JavaScript trabalham juntos para transformar uma ideia em uma aplicação real.

👏 Você não apenas construiu um player de audiobook, mas também deu os primeiros passos para dominar as bases do desenvolvimento web.



Parte Bônus

Vídeos

Complementares

Vídeos Complementares



Curso Completo de HTML: Criando a Estrutura de uma Landing Page de App de...

Hashtag Dev • 270 views • 2 months ago

[Curso Completo de HTML: Criando a Estrutura de uma Landing Page de App de Foco](#)



O Que É DOM no JavaScript e Como Manipular Elementos (Na Prática)

Hashtag Dev • 374 views • 3 weeks ago

[O Que É DOM no JavaScript e Como Manipular Elementos \(Na Prática\)](#)



Aprenda Eventos no JavaScript em 12 Minutos (Click, Input, Scroll e mais)

Hashtag Dev • 653 views • 3 months ago

[JavaScript - YouTube](#)

Ainda não segue a gente no Instagram e nem é inscrito no
nosso canal do Youtube? Então corre lá!



[@hashtagprogramacao](https://www.instagram.com/hashtagprogramacao)



youtube.com/@HashtagProgramacao

youtube.com/@DevHashtag

